



in collaboration with



Assignment Title:

Lab 2:
C Programming Basics Lab Report and Knowledge Check

Course name:

BSc. (Hons) Computer Science with AI, Software College of IT and E-Commerce,
Coventry University

CourseWork Type:

Individual

Module Name:

ST5003CMD Operating Systems, Security, and Networks

Intake: September

Submitted By:

CU ID: 16536799

College ID: 250073

Name: Ashish Shrestha(A)

Submitted To:

Rupak Rajbanshi

Table of Contents:

Section	Page
Introduction:	4
II. C Program Structure and Examples:	5
A. Basic Hello World Program:	5
B. Program with User Input:	6
C. Program with Formatted Output:	6
1. GCC Compilation Process:	9
A. Installing GCC on Linux:	9
B. Four Stages of Compilation:	9
Stage 1: Preprocessing:	10
Stage 2: Compilation to Assembly:	13
Stage 3: Assembly to Object File:	14
Stage 4: Linking:	15
IV. Return Values and Exit Status:	16
A. Understanding Return Values:	16
B. Step 2: Compile and Run:	16
B. Try with Return 1:	17
V. Knowledge Check Questions:	18
A. Question 1:	18
B. Question 2:	19
1. Preprocessing:	19
2. Assembly:	21
3. Assembler:	23
4. Linking:	24
A. Comparison Table for Four Stages of Compilation:	24
1. Value of 0 Returned by the Function:	26
B. Step 2: Compile and Run:	26
2. Non-zero Value Returned by the Function:	27
B. Step 2: Compile and Run:	27
Question 4:	28
C. Program with Integer Variable Input:	28
i. Declares an Integer Variable:	28
B. Step 2: Compile and Run:	29
iii. Displays the Entered Value Back to the User:	29
1. Preprocessing:	30
C. Program C Code for Processing:	30
B. Step 2: Compile and Run:	30
2. Compilation:	31
C. Program C Code for Compilation:	31
B. Step 2: Compile and Run:	31
3. Creation of Object File:	32
C. Program C Code for Creation of Object File:	32
B. Step 2: Compile and Run:	32
F. Question 6:	33
VI. Lab Tasks Completed:	33
1. C as a Compiled Language:	33

2. Interpreted Languages:	35
Task 1: Compiled hello.c with return 0 and verified exit status using echo \$?:	36
Task 2: Modified program to return 1 and observed different exit status:	37
Task 3: Created program accepting user input and displaying it:	37
Task 4: Executed all four compilation stages separately and documented output files:	38
Task 5: Compiled and ran program returning integer value 50:	40
VII. Screenshots and Evidence:	42
1. Update Your Package List:	42
2. Installation of GCC and Other Build Essentials:	42
3. Installation of GCC:	43
4. Check for Version:	43
5. Preprocessing Stage:	44
3. Compilation Stage:	45
4. Assembly Stage:	46
5. Program Execution with Return Value 0:	47
6. Program Execution with Return Value 1:	48
7. Program Returning Integer Value 50:	49
8. User Input Program:	50
Conclusion:	51

List of Figures:

Figure Caption	Page
Fig1: Basic Hello World Program:	5
Fig2: Command to print "Hello, World" in C:	5
Fig3: Program in C for User integer value Input:	6
Fig4: Command to return back to the user as integer value:	6
Fig5: Program in C to take input and return back to the user like name, age, and height:	7
Fig6: Command to run C code and return values like Name, Age and Height back to the user:	8
Fig7: GCC installation command:	9
Fig8: Table for stages of C compilation:	9
Fig9: Preprocessing Stage:	10
Fig10: Preprocessing stage output:	11
Fig11: Code structure for name, age, height as user input:	12
Fig12: Command to show Formatted output:	13
Fig13: Assembly Stage:	14
Fig14: Linking Stage:	15
Fig15: Program in C to return 0 as output:	16
Fig16: Program to return 0 as output.:	16
Fig17: Program in C to return 1 as output:	17
Fig18: Program to return 1 as output:	17
Fig19: Program in C that uses #include <stdio.h>:	18
Fig20: Program in C that uses both #include <stdio.h> and #define NUM function: ...	19
Fig21: Command for C program that gives output for use of "#include <stdio.h>" and #define the "NUM" function:	20
Fig22: Program in C that returns the Predefined Input:	21
Fig23: Command that returns the Predefined input integer value:	21
Fig24: Command that creates objects:	22
Fig25: Program in C that Predefines Value for the user:	23
Fig26: Command that returns Predefined value back to the user:	23
Fig27: Comparison table for Four stages of compilation:	24
Fig28: Flow Diagram for Compilation in C program:	25
Fig29: Program in C to return 0 Value:	26
Fig30: Command that shows the 0 as output using echo \$? Command:	26
Fig31: Program in C to return 1 Value:	27
Fig32: Command that shows the 1 as output using echo \$? Command:	27
Fig33: Program in C that takes integers as User input:	28
Fig34: Variable that stores Integer values as user input:	28
Fig35: Command that takes user input:	29
Fig36: Command that returns User input back to user as integer value:	29
Fig37: Program in C for processing state and create object as .i:	30
Fig38: Command in C program that returns output to the user from the object:	30
Fig39: Program in C to create objects using the .s command:	31
Fig40: Command that creates objects -o hello_word.s for Processing State:	31
Fig41: Program in C for Creation of Object File For Processing State:	32
Fig42: Command that Creates objects for processing in C:	32

Fig43: Program C code for Creation of Object File Using gcc Command:	33
Fig44: Program in C for Creation of Object File:	33
Fig45: Command to Compile a C program using GCC:	34
Fig46: Program python:	35
Fig47: Command that returns value in python Code as output:	35
Fig48: Program in C that returns 0:	36
Fig49: Command that returns 0 as output in C:	36
Fig50: C program to return value 1:	37
Fig51: Command that returns 1 as output in C:	37
Fig52: C program for user input and return the value:	38
Fig53: Command that takes integers as input and returns back to the user:	38
Fig54: C program for user input and return the value:	39
Fig55: Command that creates objects and returns output back to the user:	39
Fig56: C program code to return integer value 50:	40
Fig57: Step of Program C code to return user input back to user:	40
Fig58: Step of Program C code to return user input back to user:	41
Fig59: Update of Linux version:	42
Fig60: Installation of build-essential:	42
Fig61: Installation of GCC:	43
Fig62: Check the version of GCC:	43
Fig63: Preprocessed file hello_world.i:	44
Fig64: Generated assembly file hello.s:.....	45
Fig65: Generated object file hello.o For Assembly Stage:	46
Fig66: Program execution and exit status 0:	47
Fig67: Program execution and exit status 1:	48

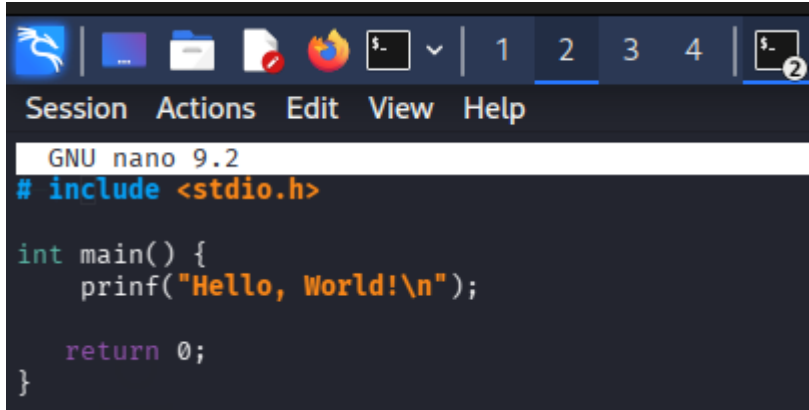
Introduction:

This report documents the practical work completed for Lecture 2 of ST5003CMD Operating Systems, Security and Networks. The focus is on understanding C program structure, compilation stages, and basic Linux commands for software development.

II. C Program Structure and examples:

A. Basic Hello World Program:

The following demonstrates a basic C program structure with preprocessor directives, main function, and return statement:

A screenshot of the nano text editor interface. The title bar shows 'GNU nano 9.2'. The menu bar includes 'Session', 'Actions', 'Edit', 'View', and 'Help'. The code content is as follows:

```
# include <stdio.h>

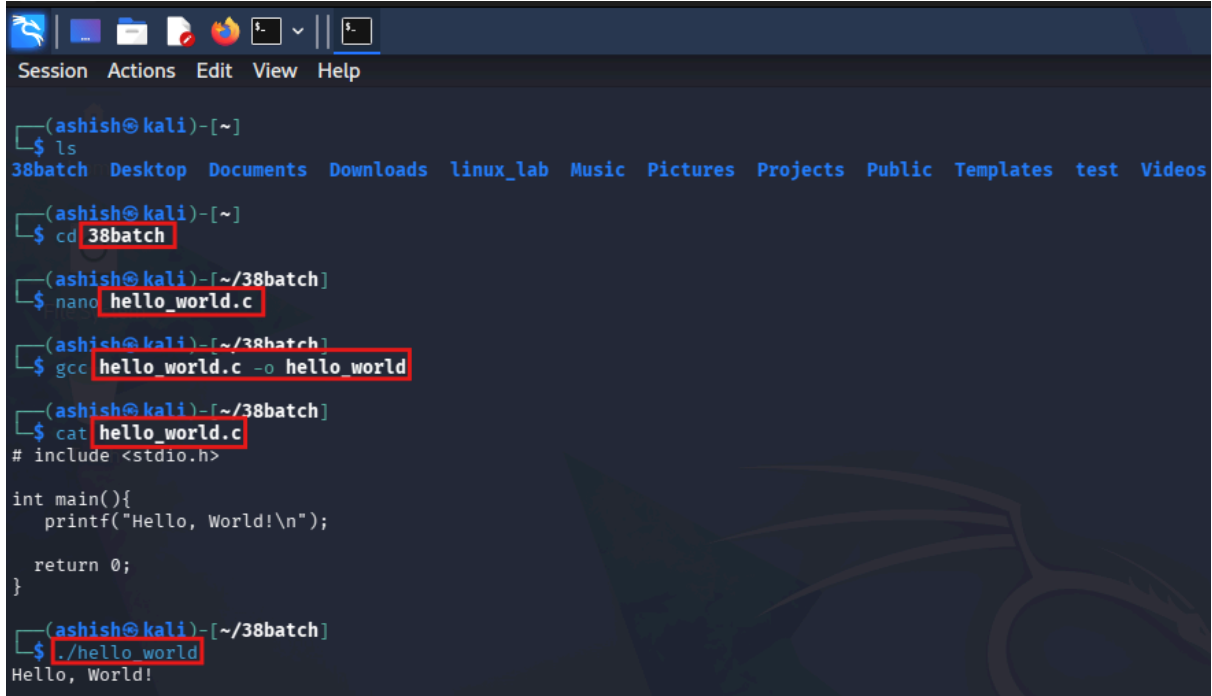
int main() {
    printf("Hello, World!\n");

    return 0;
}
```

Fig1: Basic Hello World Program.

1.File Operation: This command returns output “Hello World” for users in the C program.

Screen short:

A screenshot of a terminal window on a Kali Linux system. The user 'ashish' is at the prompt. The terminal shows the following sequence of commands and their outputs:

```
(ashish@kali)-[~]
$ ls
38batch Desktop Documents Downloads linux_lab Music Pictures Projects Public Templates test Videos

(ashish@kali)-[~]
$ cd 38batch

(ashish@kali)-[~/38batch]
$ nano hello_world.c

(ashish@kali)-[~/38batch]
$ gcc hello_world.c -o hello_world

(ashish@kali)-[~/38batch]
$ cat hello_world.c
# include <stdio.h>

int main(){
    printf("Hello, World!\n");

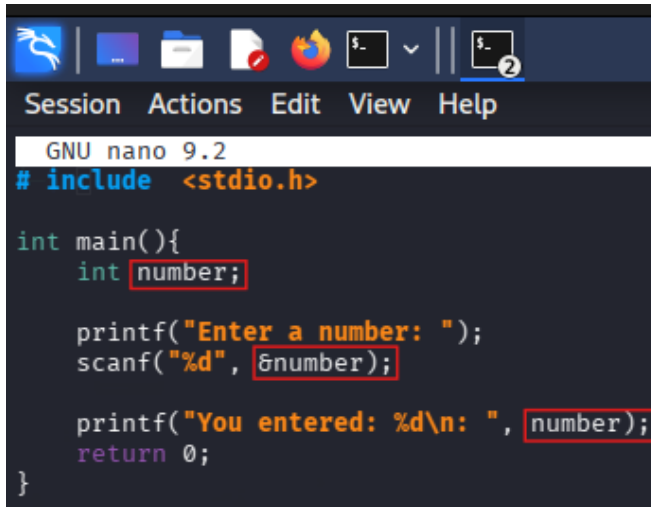
    return 0;
}

(ashish@kali)-[~/38batch]
$ ./hello_world
Hello, World!
```

Fig2: Command to print “Hello, World in C.

B. Program with User Input:

This example demonstrates the use of `printf` and `scanf` for input/output operations:

A screenshot of a terminal window with a dark background. At the top, there's a menu bar with 'Session', 'Actions', 'Edit', 'View', and 'Help'. Below it, the title bar says 'GNU nano 9.2'. The main area shows a C program. The code is:

```
#include <stdio.h>

int main(){
    int number;

    printf("Enter a number: ");
    scanf("%d", &number);

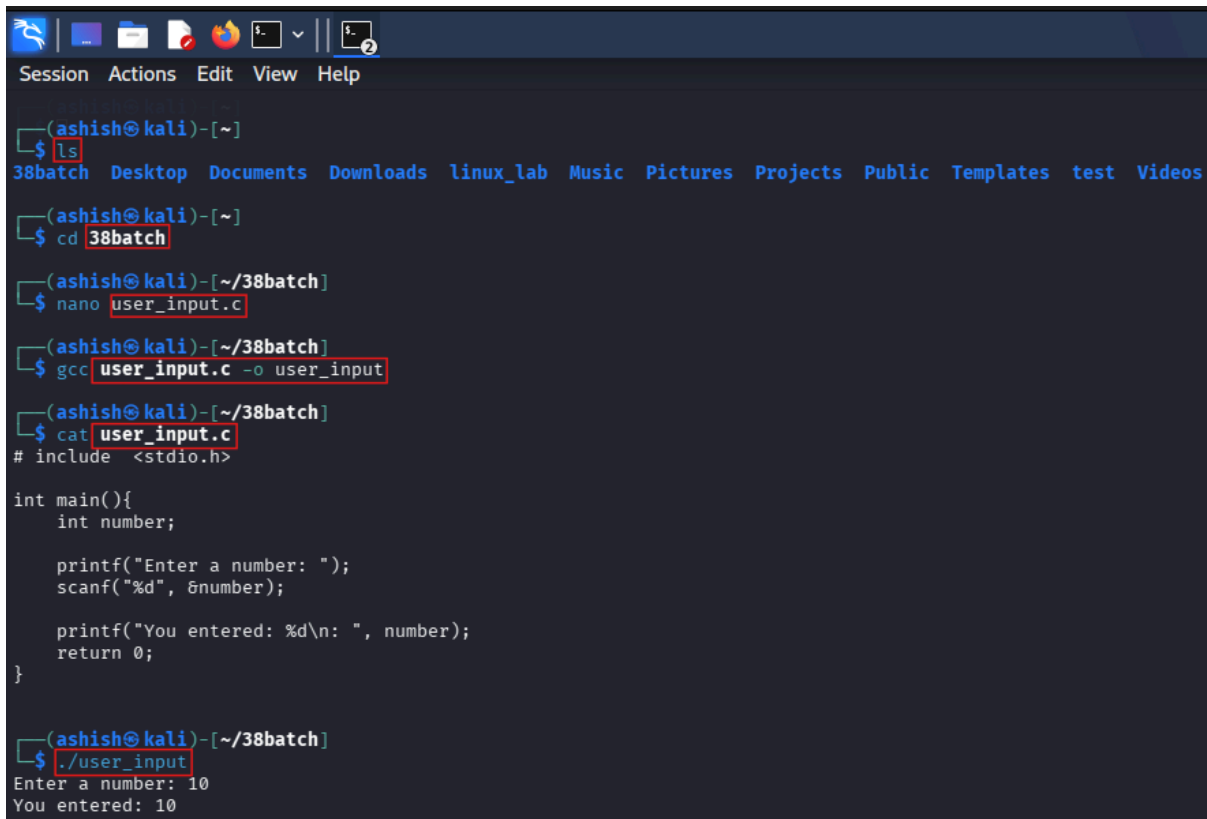
    printf("You entered: %d\n: ", number);
    return 0;
}
```

 The words 'Enter a number:' and 'You entered: %d\n: ' are in orange. The variable 'number' and its address '&number' are in red and highlighted with red boxes.

Fig3: Program in C for User integer value Input.

B. Step 2: Compile and Run: This command takes 10 as user input and returns back to the user.

Screen short:

A screenshot of a terminal window showing the steps to compile and run the program. The prompt is '(ashish@kali)-[~]'. The commands and their outputs are:

```
$ ls
38batch Desktop Documents Downloads linux_lab Music Pictures Projects Public Templates test Videos

$ cd 38batch

$ nano user_input.c

$ gcc user_input.c -o user_input

$ cat user_input.c
# include <stdio.h>

int main(){
    int number;

    printf("Enter a number: ");
    scanf("%d", &number);

    printf("You entered: %d\n: ", number);
    return 0;
}

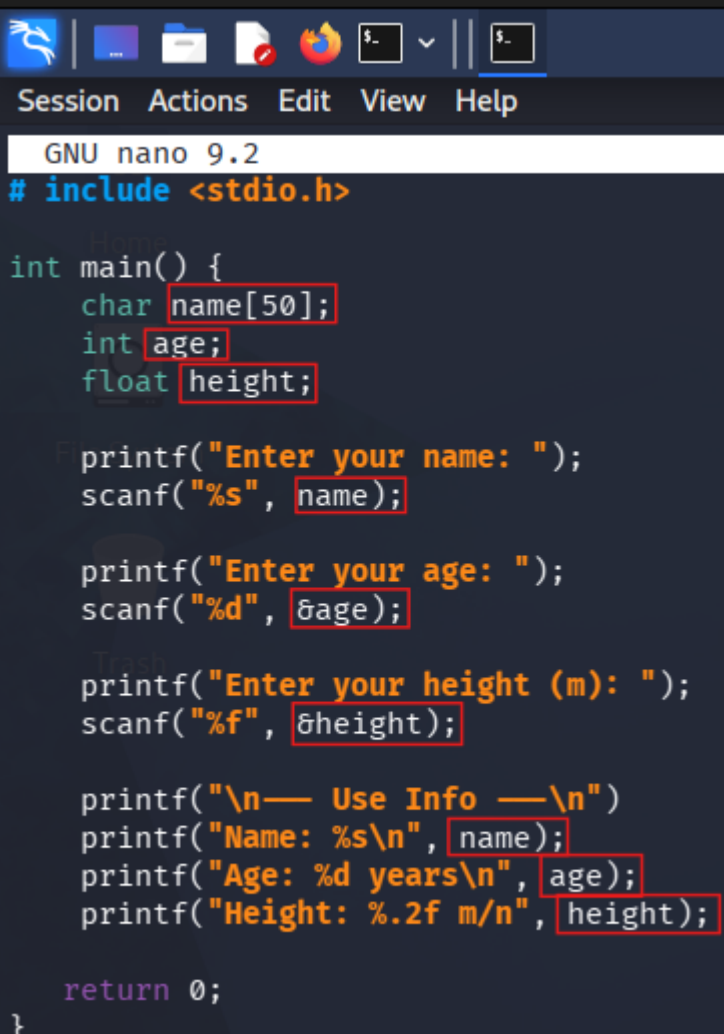
$ ./user_input
Enter a number: 10
You entered: 10
```

 The file names 'user_input.c' and 'user_input' are in red and highlighted with red boxes. The input '10' is also in red.

Fig4: Command to return back to the user as integer value.

C. Program with Formatted Output:

Demonstrating multiple format specifiers and variables:



```
GNU nano 9.2
# include <stdio.h>

int main() {
    char name[50];
    int age;
    float height;

    printf("Enter your name: ");
    scanf("%s", name);

    printf("Enter your age: ");
    scanf("%d", &age);

    printf("Enter your height (m): ");
    scanf("%f", &height);

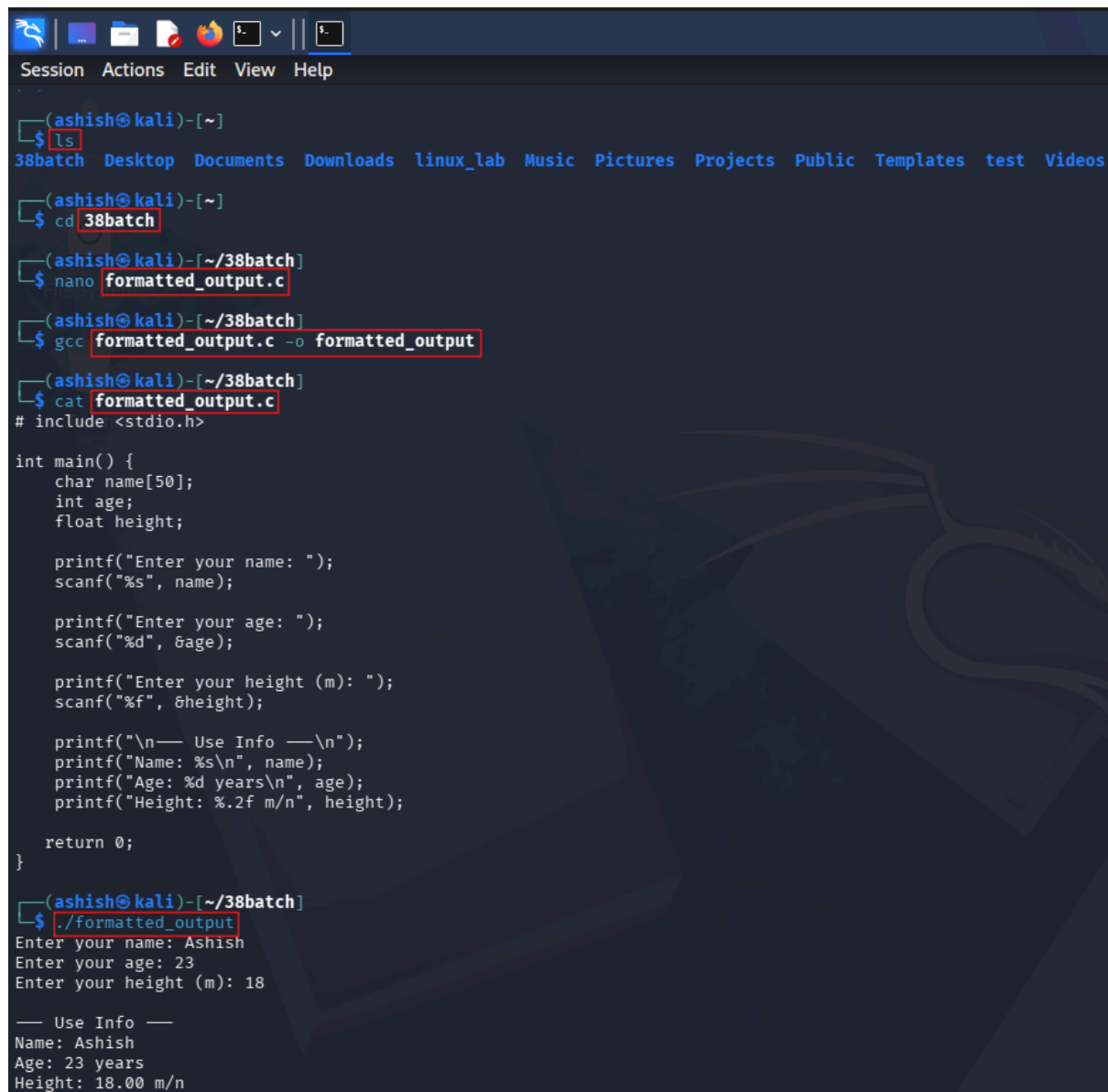
    printf("\n— Use Info —\n");
    printf("Name: %s\n", name);
    printf("Age: %d years\n", age);
    printf("Height: %.2f m/n", height);

    return 0;
}
```

Fig5: Program in C to take input and return back to the user like name, age, and height.

B. Step 2: Compile and Run: Command That takes user input as Name, Age, and Height and returns output back to the user.

Screen short:



```
(ashish@kali)-[~]
$ ls
38batch Desktop Documents Downloads linux_lab Music Pictures Projects Public Templates test Videos

(ashish@kali)-[~]
$ cd 38batch

(ashish@kali)-[~/38batch]
$ nano formatted_output.c

(ashish@kali)-[~/38batch]
$ gcc formatted_output.c -o formatted_output

(ashish@kali)-[~/38batch]
$ cat formatted_output.c
# include <stdio.h>

int main() {
    char name[50];
    int age;
    float height;

    printf("Enter your name: ");
    scanf("%s", name);

    printf("Enter your age: ");
    scanf("%d", &age);

    printf("Enter your height (m): ");
    scanf("%f", &height);

    printf("\n— Use Info —\n");
    printf("Name: %s\n", name);
    printf("Age: %d years\n", age);
    printf("Height: %.2f m/n", height);

    return 0;
}

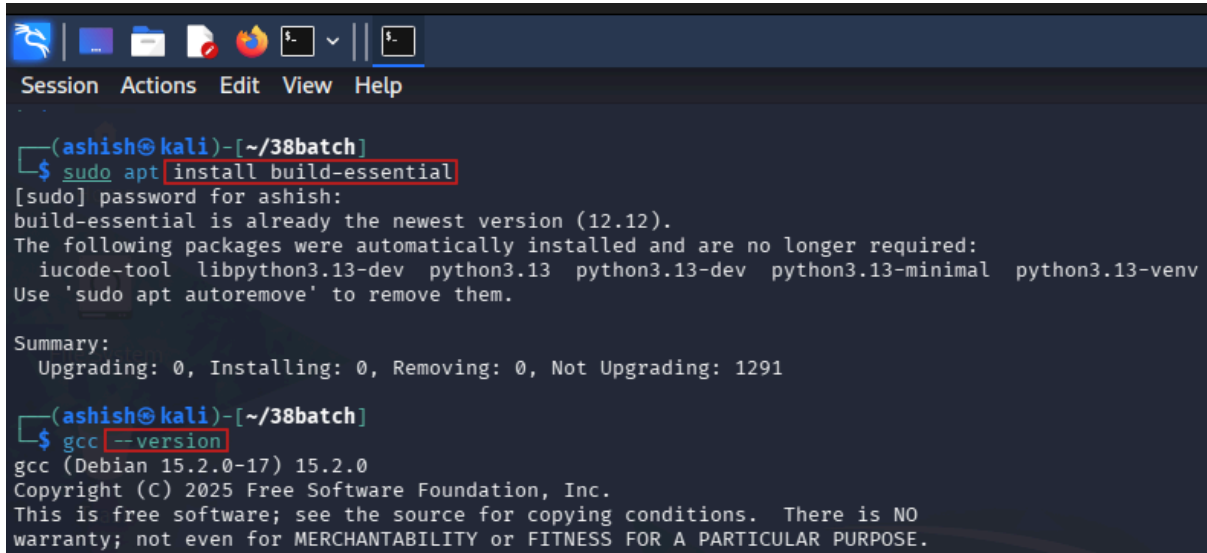
(ashish@kali)-[~/38batch]
$ ./formatted_output
Enter your name: Ashish
Enter your age: 23
Enter your height (m): 18

— Use Info —
Name: Ashish
Age: 23 years
Height: 18.00 m/n
```

Fig6: Command to run C code and return values like Name, Age and Height back to the user.

III. GCC COMPILATION PROCESS A.

Installing GCC on Linux The following commands are used to install and verify GCC:

A terminal window from Kali Linux showing the installation of build-essential and the verification of GCC version. The user 'ashish' is in the directory '~/.38batch'. The first command is 'sudo apt install build-essential', which shows that build-essential is already the newest version (12.12) and lists several packages that were automatically installed and are no longer required. The second command is 'gcc --version', which outputs 'gcc (Debian 15.2.0-17) 15.2.0' and the copyright notice for 2025 Free Software Foundation, Inc.

```
(ashish@kali)-[~/38batch]
$ sudo apt install build-essential
[sudo] password for ashish:
build-essential is already the newest version (12.12).
The following packages were automatically installed and are no longer required:
  iucode-tool libpython3.13-dev python3.13 python3.13-dev python3.13-minimal python3.13-venv
Use 'sudo apt autoremove' to remove them.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 1291

(ashish@kali)-[~/38batch]
$ gcc --version
gcc (Debian 15.2.0-17) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

Fig7: GCC installation command.

B. Four Stages of Compilation:

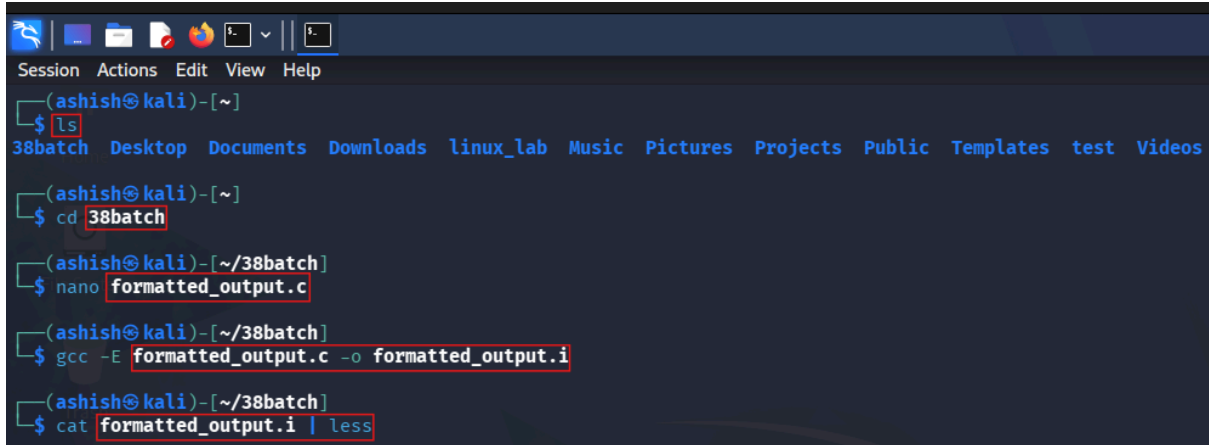
The C compilation process consists of four distinct stages. Each stage transforms the source code closer to an executable program. TABLE I THE FOUR STAGES OF C COMPILATION

THE FOUR STAGES OF C COMPILATION			
Stage	Command	Output	Purpose
1. Preprocessing	gcc -E hello.c -o hello.i	.i file	Expand headers & macros
2. Compiling	gcc -S hello.i -o hello.s	.s file	Generate Assembly code
3. Assembling	gcc -S hello.s -o hello.o	.o file	Create object file
4. Linking	gcc hello.o -o hello	executable	Final executable

Fig8: Table for stages of C compilation.

C. COMPILATION STAGE EXAMPLES 1)

Stage 1: Preprocessing:



```
(ashish@kali)~  
$ ls  
38batch Desktop Documents Downloads linux_lab Music Pictures Projects Public Templates test Videos  
(ashish@kali)~  
$ cd 38batch  
(ashish@kali)~/38batch  
$ nano formatted_output.c  
(ashish@kali)~/38batch  
$ gcc -E formatted_output.c -o formatted_output.i  
(ashish@kali)~/38batch  
$ cat formatted_output.i | less
```

Fig9: Preprocessing Stage.

Output:



```
# 0 "formatted_output.c"
# 0 "<built-in>"
# 0 "<command-line>"
# 1 "/usr/include/stdc-predef.h" 1 3 4
# 0 "<command-line>" 2
# 1 "formatted_output.c"
# 1 "/usr/include/stdio.h" 1 3 4
# 28 "/usr/include/stdio.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 1 3 4
# 33 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 3 4
# 1 "/usr/include/features.h" 1 3 4
# 415 "/usr/include/features.h" 3 4
# 1 "/usr/include/features-time64.h" 1 3 4
# 20 "/usr/include/features-time64.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 21 "/usr/include/features-time64.h" 2 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 1 3 4
# 19 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 20 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 2 3 4
# 22 "/usr/include/features-time64.h" 2 3 4
# 416 "/usr/include/features.h" 2 3 4
# 523 "/usr/include/features.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 1 3 4
# 730 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 731 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 2 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/long-double.h" 1 3 4
# 732 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 2 3 4
# 524 "/usr/include/features.h" 2 3 4
# 547 "/usr/include/features.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/gnu/stubs.h" 1 3 4
# 10 "/usr/include/x86_64-linux-gnu/gnu/stubs.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/gnu/stubs-64.h" 1 3 4
# 11 "/usr/include/x86_64-linux-gnu/gnu/stubs.h" 2 3 4
# 548 "/usr/include/features.h" 2 3 4
# 34 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 2 3 4
# 29 "/usr/include/stdio.h" 2 3 4
```

Fig10: Preprocessing stage output.

Output:

```
# 2 "formatted_output.c" 2

# 3 "formatted_output.c"
int main() {
    char name[50];
    int age;
    float height;

    printf("Enter your name: ");
    scanf("%s", name);

    printf("Enter your age: ");
    scanf("%d", &age);

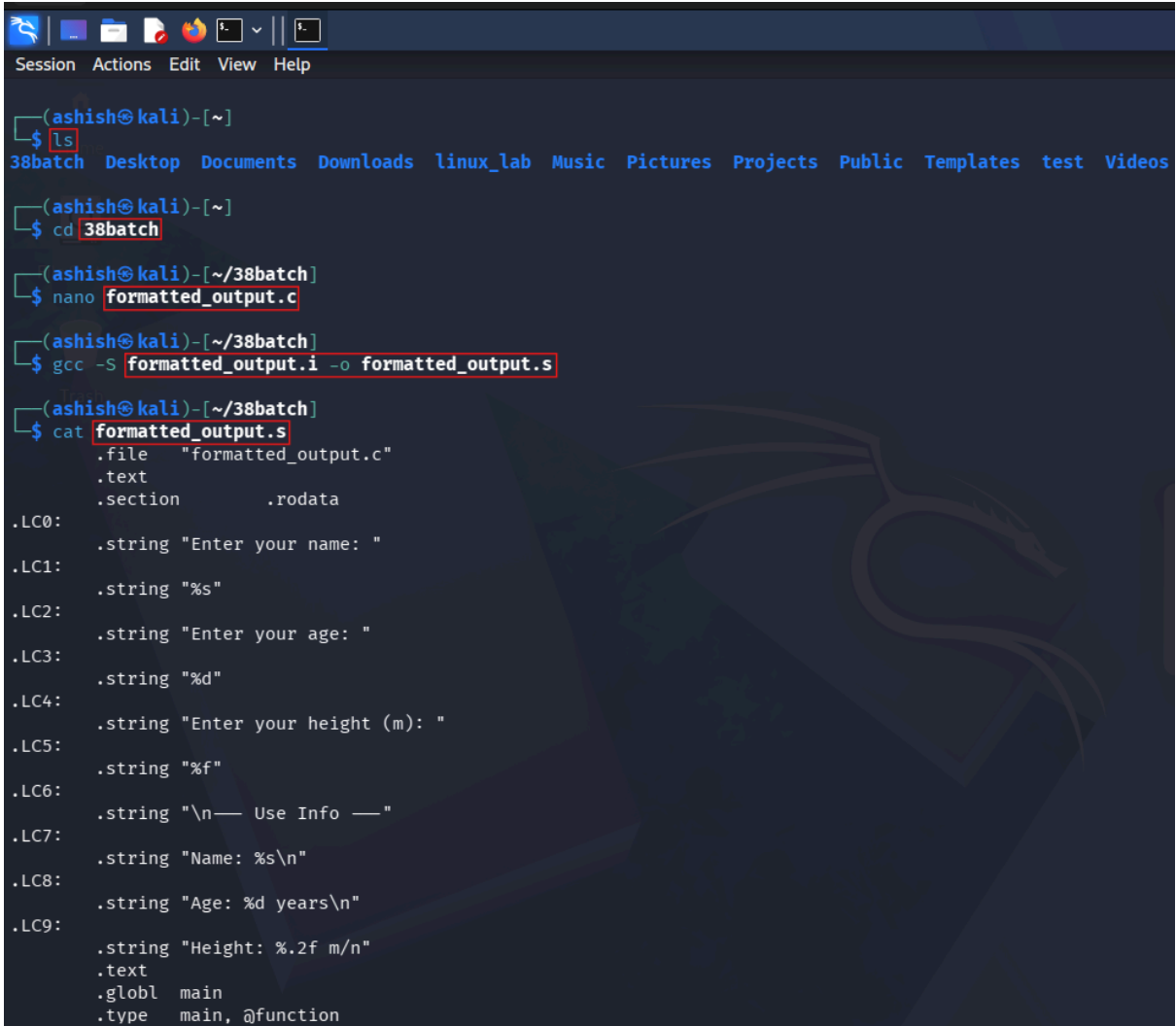
    printf("Enter your height (m): ");
    scanf("%f", &height);

    printf("\n— Use Info —\n");
    printf("Name: %s\n", name);
    printf("Age: %d years\n", age);
    printf("Height: %.2f m/n", height);

    return 0;
}
```

Fig11: Code structure for name, age, height as user input.

2) Stage 2: Compilation to Assembly:



```
(ashish@kali)-[~]
$ ls
38batch Desktop Documents Downloads linux_lab Music Pictures Projects Public Templates test Videos

(ashish@kali)-[~]
$ cd 38batch

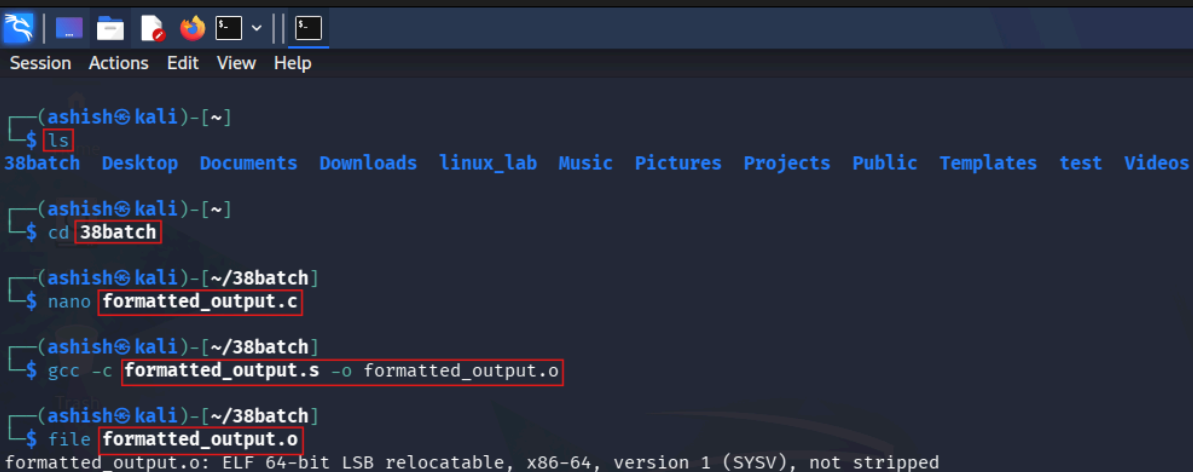
(ashish@kali)-[~/38batch]
$ nano formatted_output.c

(ashish@kali)-[~/38batch]
$ gcc -S formatted_output.i -o formatted_output.s

(ashish@kali)-[~/38batch]
$ cat formatted_output.s
.file "formatted_output.c"
.text
.section .rodata
.LC0:
.string "Enter your name: "
.LC1:
.string "%s"
.LC2:
.string "Enter your age: "
.LC3:
.string "%d"
.LC4:
.string "Enter your height (m): "
.LC5:
.string "%f"
.LC6:
.string "\n— Use Info —"
.LC7:
.string "Name: %s\n"
.LC8:
.string "Age: %d years\n"
.LC9:
.string "Height: %.2f m/n"
.text
.globl main
.type main, @function
```

Fig12: Command to to show Formatted output.

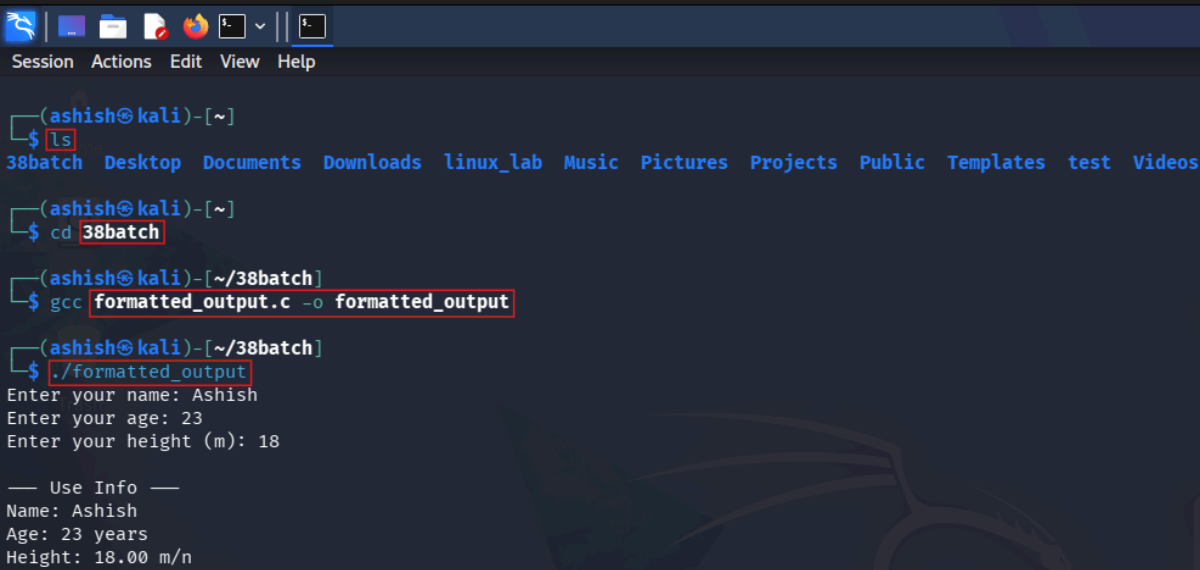
3) Stage 3: Assembly to Object File:



```
(ashish@kali)~  
$ ls  
38batch Desktop Documents Downloads linux_lab Music Pictures Projects Public Templates test Videos  
(ashish@kali)~  
$ cd 38batch  
(ashish@kali)~/38batch  
$ nano formatted_output.c  
(ashish@kali)~/38batch  
$ gcc -c formatted_output.s -o formatted_output.o  
(ashish@kali)~/38batch  
$ file formatted_output.o  
formatted_output.o: ELF 64-bit LSB relocatable, x86-64, version 1 (SYSV), not stripped
```

Fig13: Assembly Stage.

4) Stage 4: Linking:



```
(ashish@kali)-[~]
$ ls
38batch Desktop Documents Downloads linux_lab Music Pictures Projects Public Templates test Videos

(ashish@kali)-[~]
$ cd 38batch

(ashish@kali)-[~/38batch]
$ gcc formatted_output.c -o formatted_output

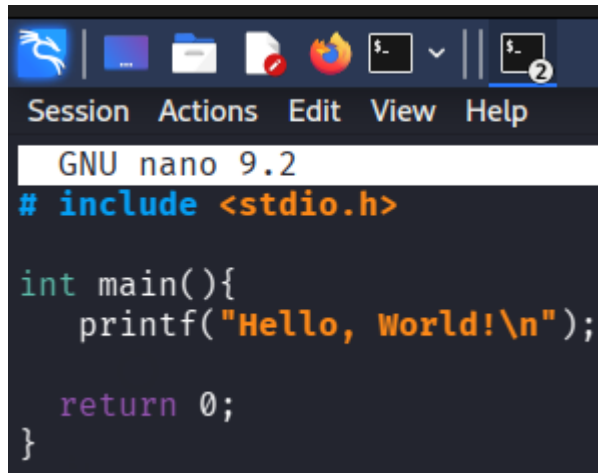
(ashish@kali)-[~/38batch]
$ ./formatted_output
Enter your name: Ashish
Enter your age: 23
Enter your height (m): 18

— Use Info —
Name: Ashish
Age: 23 years
Height: 18.00 m/n
```

Fig14: Linking Stage.

IV. RETURN VALUES AND EXIT STATUS A.

Understanding Return Values The return value of **main()** indicates program success or failure to the OS:

A screenshot of a nano text editor window. The title bar shows icons for a terminal, file manager, and other applications. The menu bar includes Session, Actions, Edit, View, and Help. The text area contains the following C code:

```
GNU nano 9.2
# include <stdio.h>

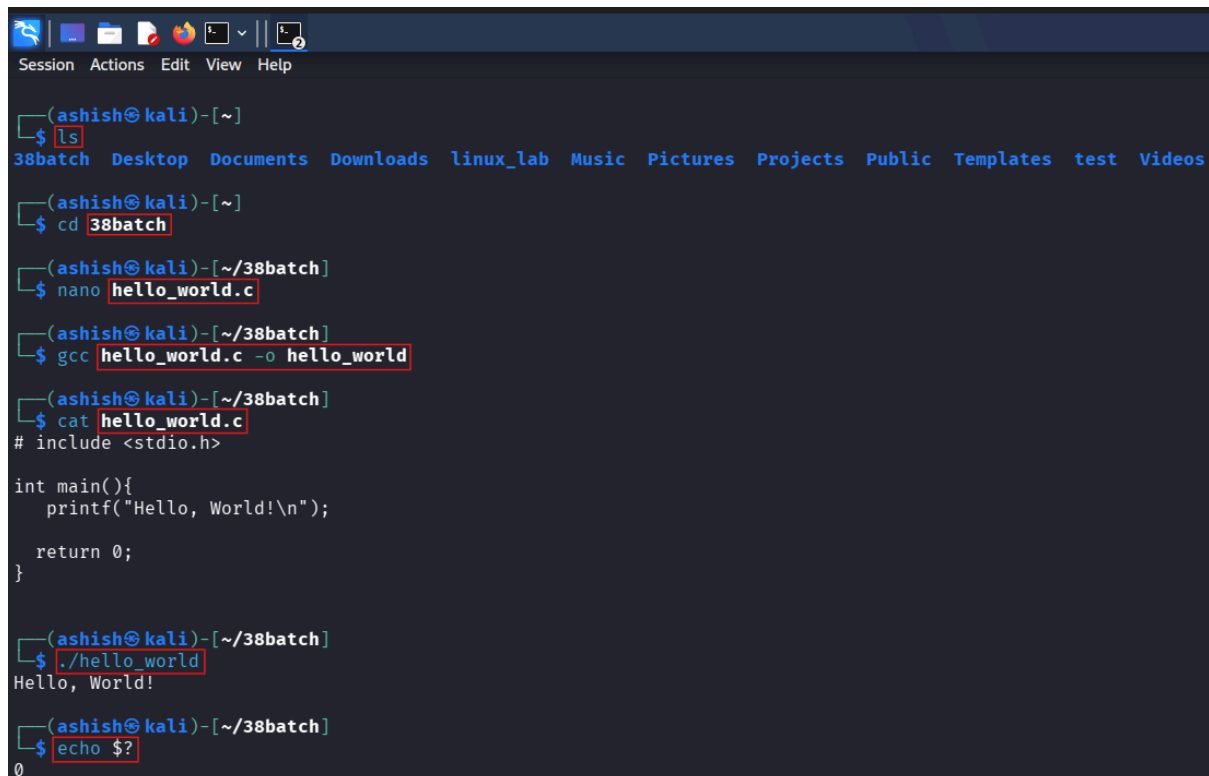
int main(){
    printf("Hello, World!\n");

    return 0;
}
```

Fig15: Program in c to return 0 as output.

B. Step 2: Compile and Run: Program that returns 0 as output.

Screen short:

A screenshot of a terminal window. The title bar shows icons for a terminal, file manager, and other applications. The menu bar includes Session, Actions, Edit, View, and Help. The terminal shows the following commands and output:

```
(ashish@kali)-[~]
$ ls
38batch Desktop Documents Downloads linux_lab Music Pictures Projects Public Templates test Videos

(ashish@kali)-[~]
$ cd 38batch

(ashish@kali)-[~/38batch]
$ nano hello_world.c

(ashish@kali)-[~/38batch]
$ gcc hello_world.c -o hello_world

(ashish@kali)-[~/38batch]
$ cat hello_world.c
# include <stdio.h>

int main(){
    printf("Hello, World!\n");

    return 0;
}

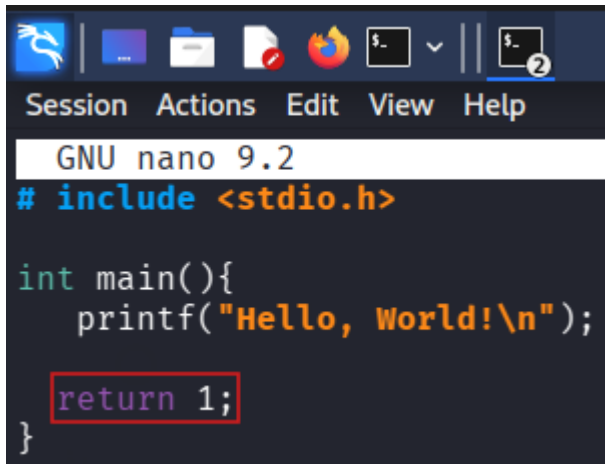
(ashish@kali)-[~/38batch]
$ ./hello_world
Hello, World!

(ashish@kali)-[~/38batch]
$ echo $?
0
```

Fig16: Program to return 0 as output.

B. Try with return 1

Modify program to return 1, recompile, and check:



```
GNU nano 9.2
# include <stdio.h>

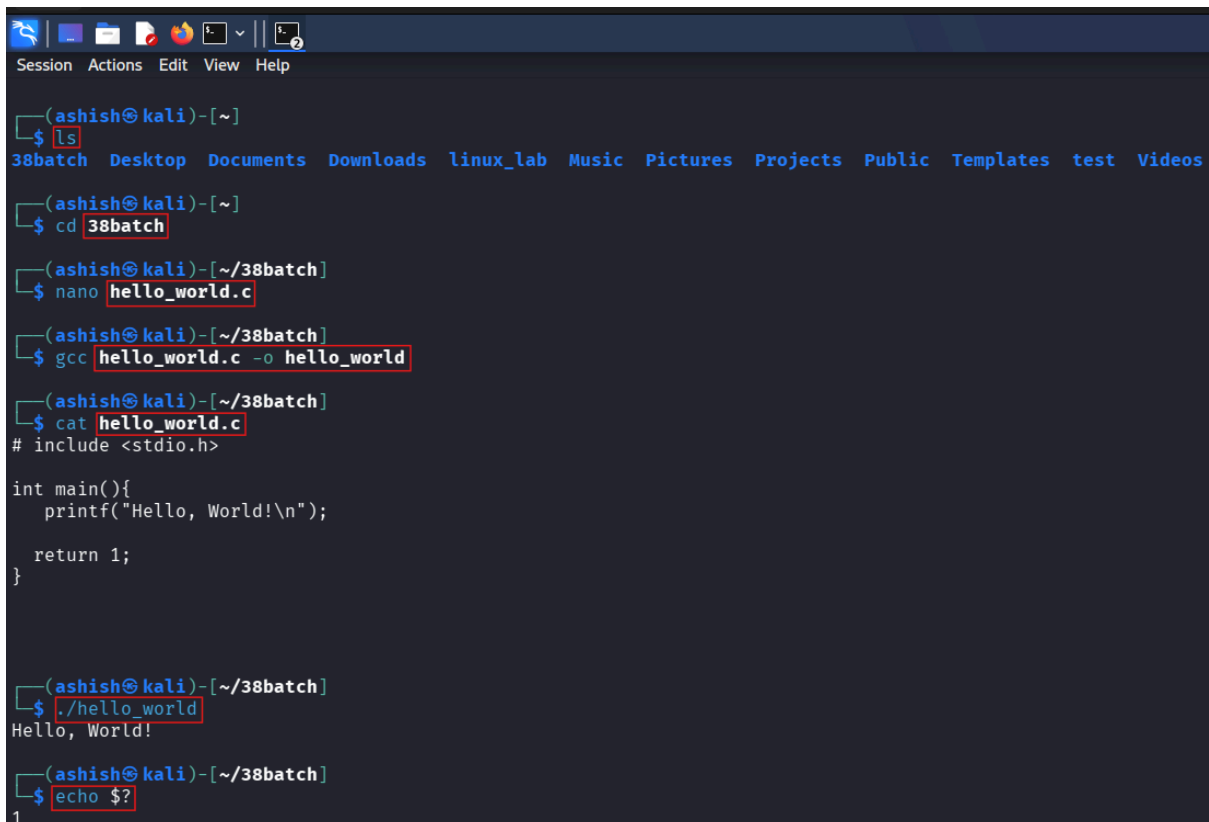
int main(){
    printf("Hello, World!\n");

    return 1;
}
```

Fig17: Program in c to return 1 as output.

B. Step 2: Compile and Run: Program that returns 1 as output.

Screen short:



```
(ashish@kali)-[~]
$ ls
38batch Desktop Documents Downloads linux_lab Music Pictures Projects Public Templates test Videos

(ashish@kali)-[~]
$ cd 38batch

(ashish@kali)-[~/38batch]
$ nano hello_world.c

(ashish@kali)-[~/38batch]
$ gcc hello_world.c -o hello_world

(ashish@kali)-[~/38batch]
$ cat hello_world.c
# include <stdio.h>

int main(){
    printf("Hello, World!\n");

    return 1;
}

(ashish@kali)-[~/38batch]
$ ./hello_world
Hello, World!

(ashish@kali)-[~/38batch]
$ echo $?
1
```

Fig18: Program to return 1 as output.

V. KNOWLEDGE CHECK QUESTIONS:

The following questions assess understanding of Lecture 2 concepts:

A. Question 1

Q: What is the purpose of the `#include` directive in C programming? Provide an example of its usage.

Summary:

The **#include** directive in C language is employed for including the contents of the header file into a C program. This way, we can utilize the predefined functions and declarations from the header file.

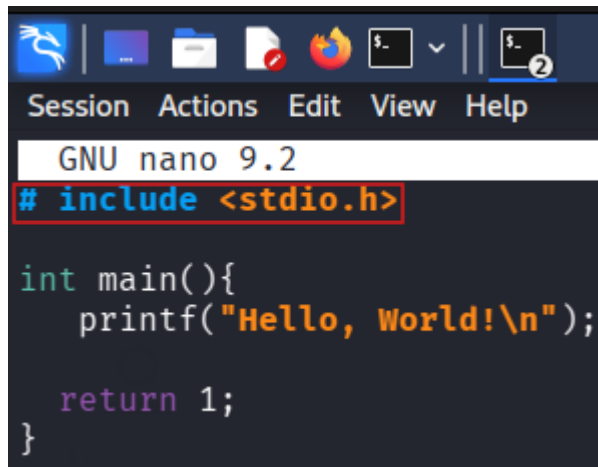
A screenshot of a GNU nano 9.2 text editor window. The window has a dark blue title bar with icons for nano, a file, a folder, a terminal, and a search icon. Below the title bar is a menu bar with 'Session', 'Actions', 'Edit', 'View', and 'Help'. The main editing area has a dark background with syntax-highlighted C code. The first line is '#include <stdio.h>' with a red box around it. The second line is 'int main(){', the third is 'printf("Hello, World!\n");', the fourth is 'return 1;', and the fifth is '}'.

Fig19: Program in C that Use `#include <stdio.h>`

Explanation:

- `#include <stdio.h>` is used for including the standard input/output header file.
- This enables us to use the `printf()` function to print on the screen.

B. Question 2 Q:

Explain the difference between the four stages of compilation. What file extension is produced at each stage?

Difference between the Four Stages of Compilation in C:

There are four major stages involved when a C program is transformed into an executable program.

1. Preprocessing:

Function:

Deals with preprocessor instructions like **#include** and **#define**.

Eliminates comments from the program.

- Inserts header files.
- Expands macros.
- Generates preprocessed source code.

Example:

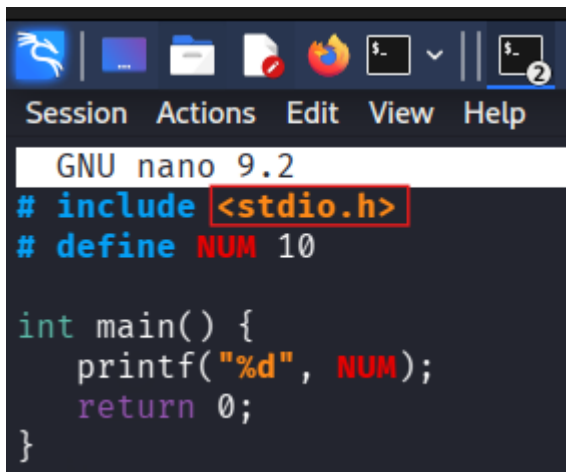
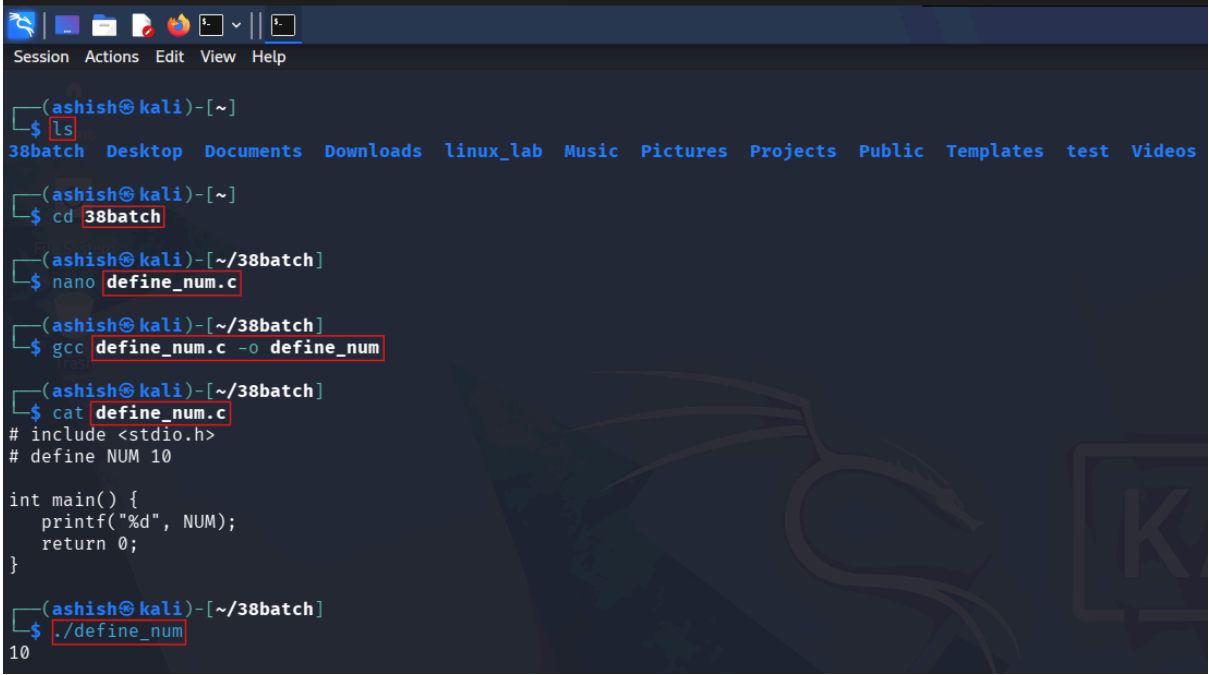
A screenshot of a GNU nano 9.2 text editor window. The window has a dark blue title bar with icons for file operations and a menu bar with 'Session', 'Actions', 'Edit', 'View', and 'Help'. The editor content is on a dark background with syntax-highlighted C code. The first two lines are preprocessor directives: '# include <stdio.h>' and '# define NUM 10'. The third line is 'int main() {', followed by an indented 'printf("%d", NUM);', then 'return 0;', and finally a closing brace '}'.

Fig20: Program in C that use both **# include <stdio.h>** and **# define NUM** Function.

B. Step 2: Compile and Run: Command that returns the predefined integer value to the user.

Screen short:



```
(ashish@kali)-[~]
$ ls
38batch Desktop Documents Downloads linux_lab Music Pictures Projects Public Templates test Videos

(ashish@kali)-[~]
$ cd 38batch

(ashish@kali)-[~/38batch]
$ nano define_num.c

(ashish@kali)-[~/38batch]
$ gcc define_num.c -o define_num

(ashish@kali)-[~/38batch]
$ cat define_num.c
# include <stdio.h>
# define NUM 10

int main() {
    printf("%d", NUM);
    return 0;
}

(ashish@kali)-[~/38batch]
$ ./define_num
10
```

Fig21: Command for c Program that give output for use of “# include <stdio.h>” and # define the “NUM” function.

Summary:

The preprocessing phase replaces “#include <stdio.h>” and “NUM” with 10.

Output file: .i

2. Assembly:

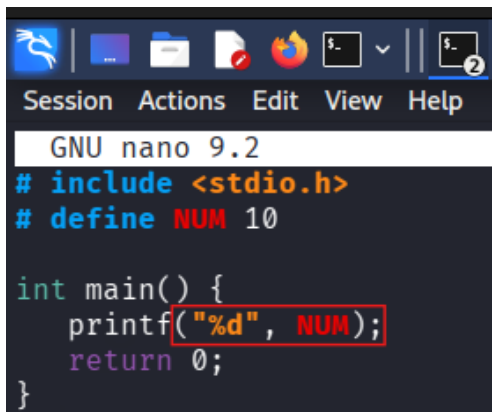
Objective:

Converts preprocessed C source code to assembly code.

- Does syntax and semantic checking on the C source code.
- Produces errors during compilation.
- Generates assembly code from the C code.

Output File: .s

Example:



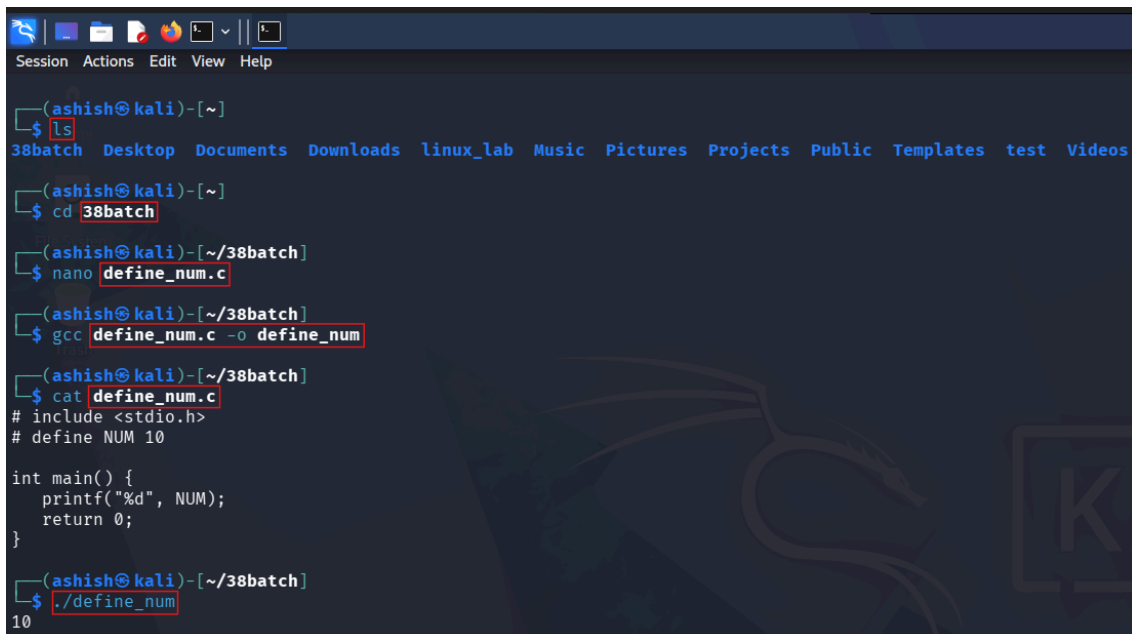
```
GNU nano 9.2
# include <stdio.h>
# define NUM 10

int main() {
    printf("%d", NUM);
    return 0;
}
```

Fig22: Program in C that returns the Predefined Input.

B. Step 2: Compile and Run: Command that returns the predefined user input to user.

Screen short:



```
(ashish@kali)-[~]
$ ls
38batch Desktop Documents Downloads linux_lab Music Pictures Projects Public Templates test Videos

(ashish@kali)-[~]
$ cd 38batch

(ashish@kali)-[~/38batch]
$ nano define_num.c

(ashish@kali)-[~/38batch]
$ gcc define_num.c -o define_num

(ashish@kali)-[~/38batch]
$ cat define_num.c
# include <stdio.h>
# define NUM 10

int main() {
    printf("%d", NUM);
    return 0;
}

(ashish@kali)-[~/38batch]
$ ./define_num
10
```

Fig23: Command that returns the Predefined input integer value.

Summary:

Translation into assembly language code for execution by the processor is done.

Object File:

An object file includes the machine language code which is generated after assembling but has not yet been linked to any other files or libraries.

➤ **GCC command to create an object:**

gcc -E program.c -o program.i

A terminal window screenshot with a dark background. The prompt is '(ashish@kali)-[~/38batch]'. The command '\$ gcc hello_world.c -o hello_world' is entered. The command and its arguments are highlighted with a red rectangular box.

```
(ashish@kali)-[~/38batch]  
$ gcc hello_world.c -o hello_world
```

Fig24: Command that creates objects.

3. Assembler:

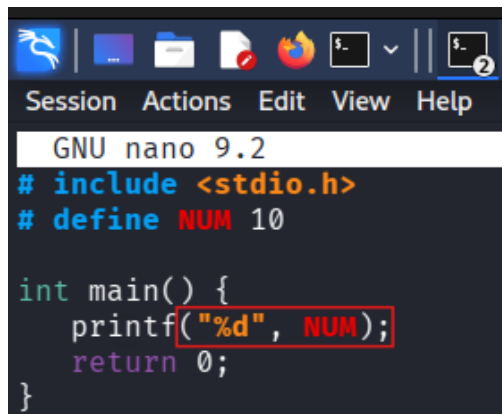
Objective:

To transform assembly language into machine language.

- Transformation of assembly language into binary machine language.
- Creation of object code.

Output file: .o

Example:



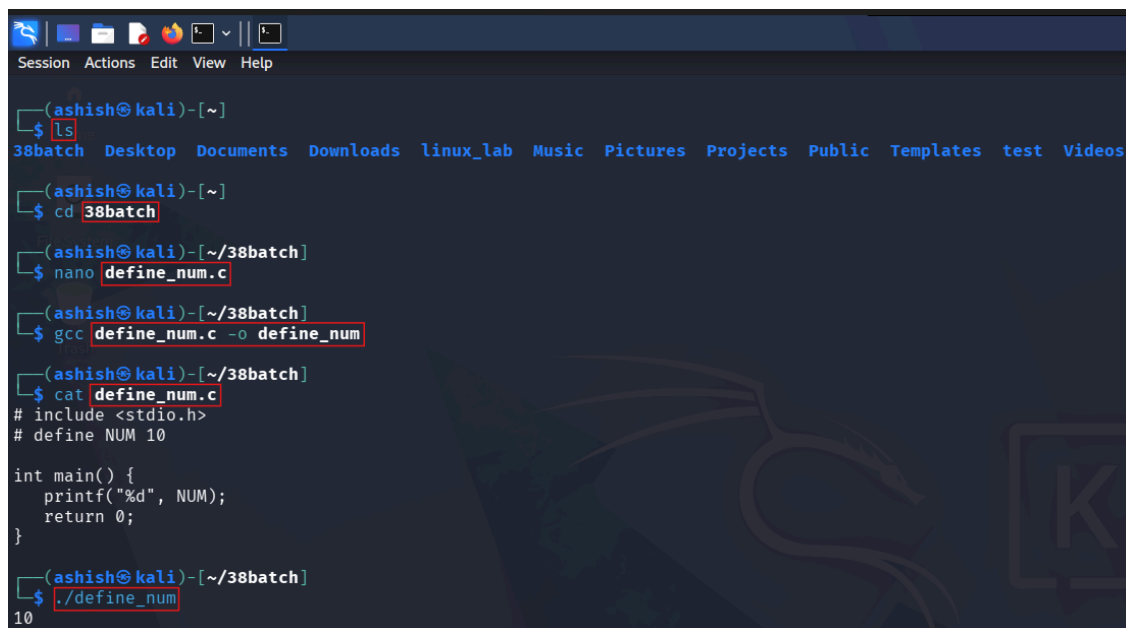
```
GNU nano 9.2
# include <stdio.h>
# define NUM 10

int main() {
    printf("%d", NUM);
    return 0;
}
```

Fig25: Program in C that Predefines Value for the user.

B. Step 2: Compile and Run: Assembler and use it to create an object.

Screen short:



```
(ashish@kali)-[~]
$ ls
38batch Desktop Documents Downloads linux_lab Music Pictures Projects Public Templates test Videos
(ashish@kali)-[~]
$ cd 38batch
(ashish@kali)-[~/38batch]
$ nano define_num.c
(ashish@kali)-[~/38batch]
$ gcc define_num.c -o define_num
(ashish@kali)-[~/38batch]
$ cat define_num.c
# include <stdio.h>
# define NUM 10

int main() {
    printf("%d", NUM);
    return 0;
}
(ashish@kali)-[~/38batch]
$ ./define_num
10
```

Fig26: Command that returns Predefined value back to user.

Summary:

The object code is not always a complete executable since there may be other functions to link with.

4. Linking:**Purpose:**

Links objects together with necessary libraries into one executable program.

- Links call of functions to the definition of these functions.
- Links library functions like printf().
- Solves external links.
- Generates the executable.

Output file:

Executable, for example, a.out, program.

A. Comparison Table for Four Stages of Compilation:

The C compilation process consists of four distinct stages. Each stage transforms the source code closer to an executable program. TABLE I THE FOUR STAGES OF C COMPILATION

<u>Comprasion Table</u>			
<u>Stage</u>	<u>Main Purpose</u>	<u>Input</u>	<u>Output</u>
Preprocessing	Handles directives and expands macros	.c	.i
Compilation	Converts C into assembly	.i	.s
Assembly	Converts assembbly Into machine code	.s	.o
Linking	Combines object code and libraries	.o	Executable

Fig27: Comparison table for Four stages of compilation.

Complete Compilation Flow:

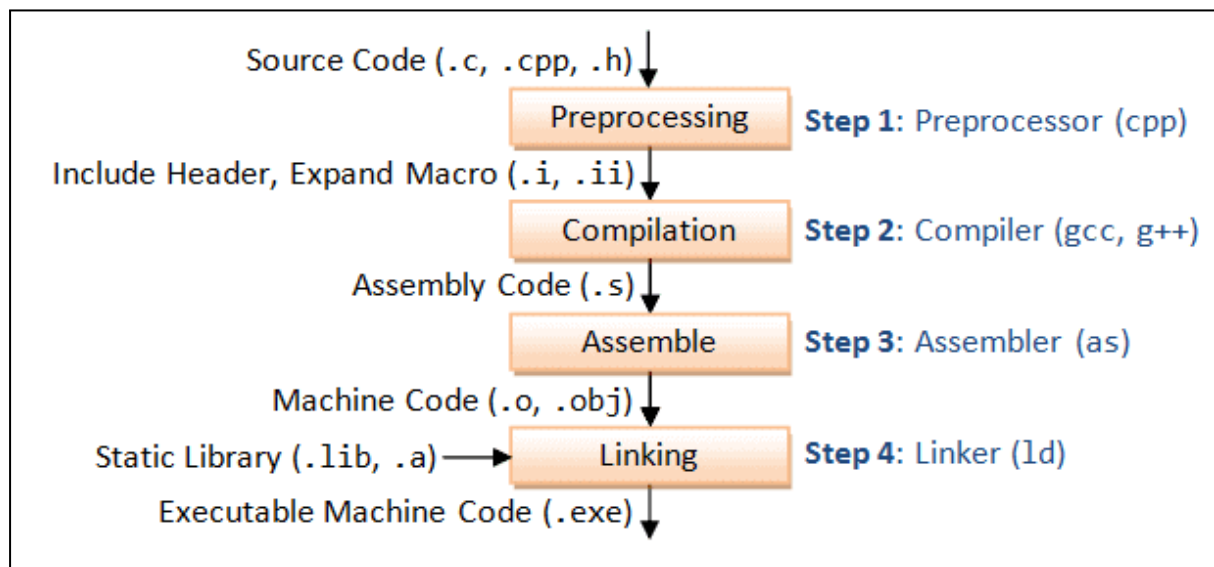


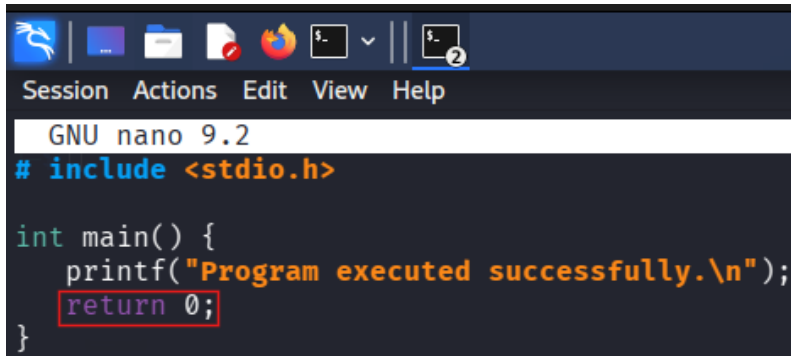
Fig28: Flow Diagram for Compilation in C program.

C. Question 3 Q:

What does a return value of 0 from the main() function indicate to the operating system? What about a non-zero return value?

1. Value of 0 returned by the function:

The function main() returns a value of 0, it means that the program ran successfully without errors to the OS.



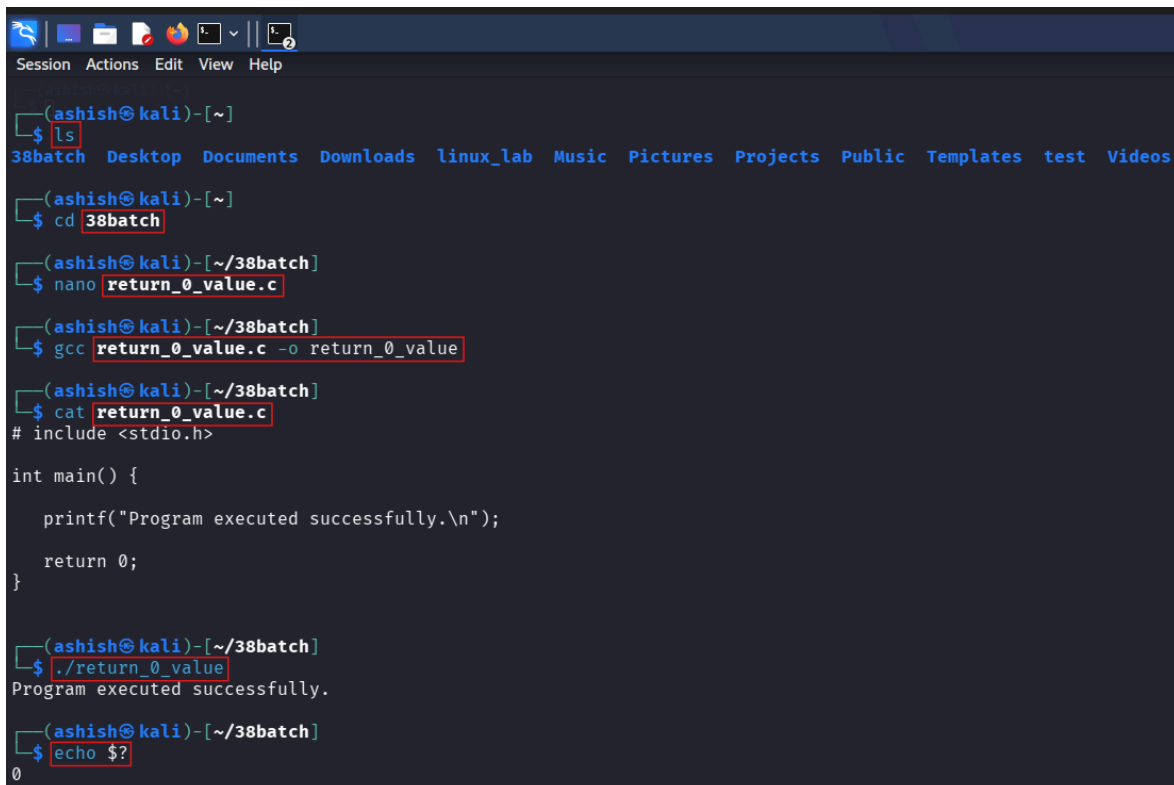
```
GNU nano 9.2
# include <stdio.h>

int main() {
    printf("Program executed successfully.\n");
    return 0;
}
```

Fig29: Program in C to return 0 Value.

B. Step 2: Compile and Run: Command that returns 0 as output using \$ echo?.

Screen short:



```
(ashish@kali)-[~]
$ ls
38batch Desktop Documents Downloads linux_lab Music Pictures Projects Public Templates test Videos

(ashish@kali)-[~]
$ cd 38batch

(ashish@kali)-[~/38batch]
$ nano return_0_value.c

(ashish@kali)-[~/38batch]
$ gcc return_0_value.c -o return_0_value

(ashish@kali)-[~/38batch]
$ cat return_0_value.c
# include <stdio.h>

int main() {

    printf("Program executed successfully.\n");

    return 0;

}

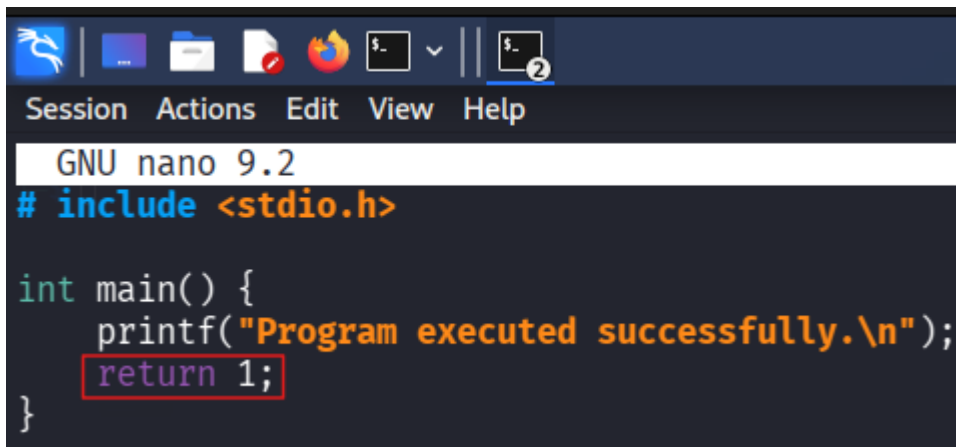
(ashish@kali)-[~/38batch]
$ ./return_0_value
Program executed successfully.

(ashish@kali)-[~/38batch]
$ echo $?
0
```

Fig30: Command that shows the 0 as output using echo \$? command.

2. Non-zero value returned by the function:

If the return value is non-zero, it means that the program did not run successfully or that there were errors in the process.



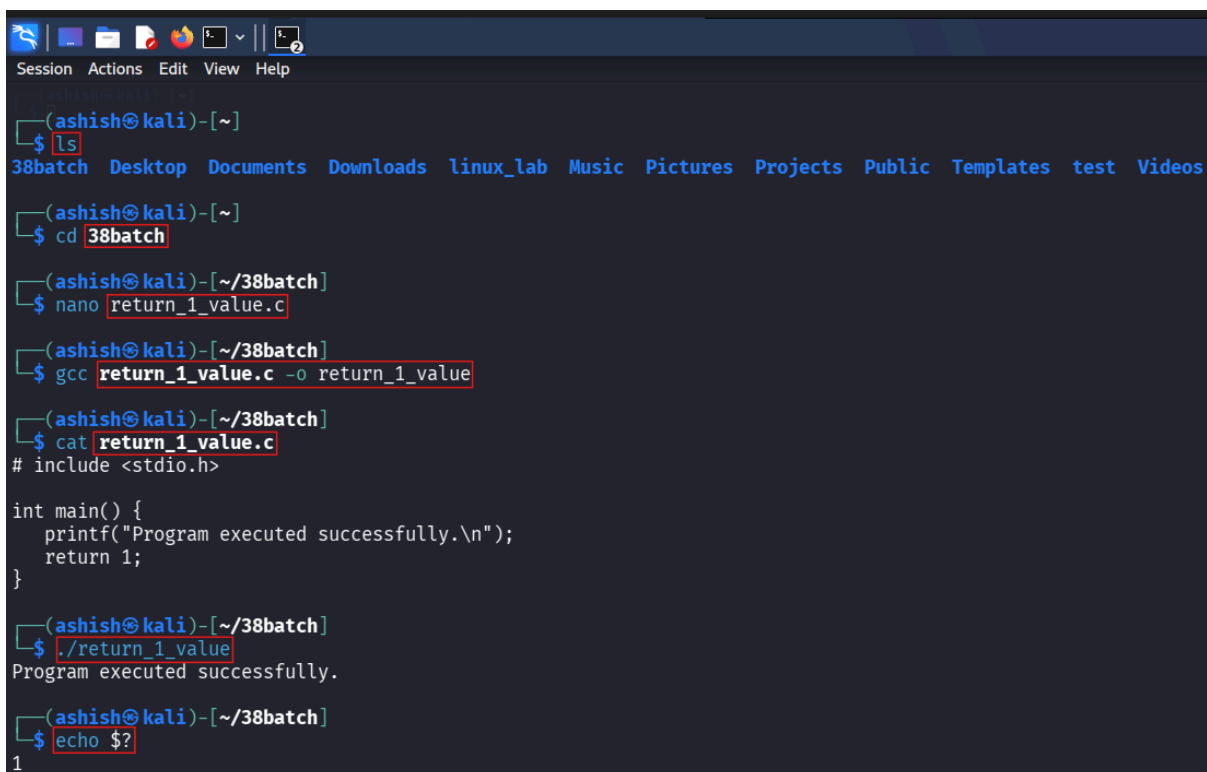
```
GNU nano 9.2
# include <stdio.h>

int main() {
    printf("Program executed successfully.\n");
    return 1;
}
```

Fig31: Program in C to return 1 Value.

B. Step 2: Compile and Run: Command that returns 1 as output using \$ echo?.

Screen short:



```
(ashish@kali)-[~]
$ ls
38batch Desktop Documents Downloads linux_lab Music Pictures Projects Public Templates test Videos

(ashish@kali)-[~]
$ cd 38batch

(ashish@kali)-[~/38batch]
$ nano return_1_value.c

(ashish@kali)-[~/38batch]
$ gcc return_1_value.c -o return_1_value

(ashish@kali)-[~/38batch]
$ cat return_1_value.c
# include <stdio.h>

int main() {
    printf("Program executed successfully.\n");
    return 1;
}

(ashish@kali)-[~/38batch]
$ ./return_1_value
Program executed successfully.

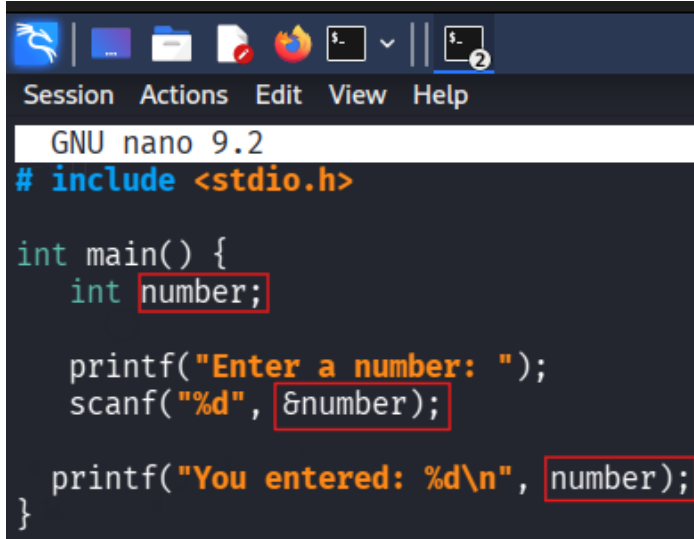
(ashish@kali)-[~/38batch]
$ echo $?
1
```

Fig32: Command that shows the 1 as output using echo \$? command.

Question 4 Q: Write a C program that:

- Declares an integer variable
- Asks the user to input a value
- Displays the entered value back to the use

C. Program with integer variable input:

A screenshot of a terminal window showing the GNU nano 9.2 text editor. The editor contains a C program that prompts the user to enter a number and then displays the entered value. The code is as follows:

```
#include <stdio.h>

int main() {
    int number;

    printf("Enter a number: ");
    scanf("%d", &number);

    printf("You entered: %d\n", number);
}
```

The variable 'number' is highlighted with a red box in the original image.

Fig33: Program in C that takes integers as User input .

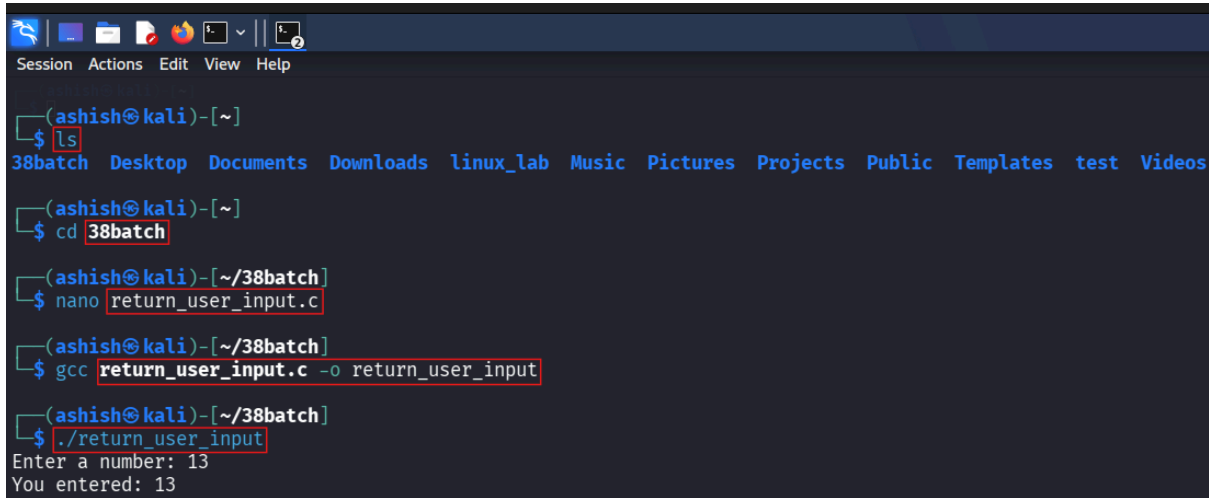
i. Declares an integer variable:

```
int number;
```

Fig34: Variable that stores Integer values as user input.

B. Step 2: Compile and Run: Command that creates objects and takes user input and returns back to the user.

Screen short:



```
(ashish@kali)-[~]
$ ls
38batch Desktop Documents Downloads linux_lab Music Pictures Projects Public Templates test Videos

(ashish@kali)-[~]
$ cd 38batch

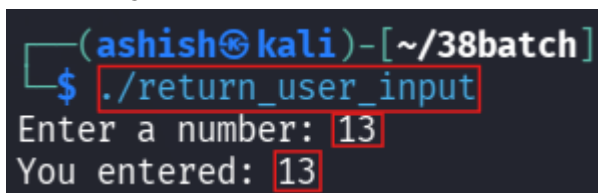
(ashish@kali)-[~/38batch]
$ nano return_user_input.c

(ashish@kali)-[~/38batch]
$ gcc return_user_input.c -o return_user_input

(ashish@kali)-[~/38batch]
$ ./return_user_input
Enter a number: 13
You entered: 13
```

Fig35: Command that takes user input.

iii. Displays the entered value back to the use:



```
(ashish@kali)-[~/38batch]
$ ./return_user_input
Enter a number: 13
You entered: 13
```

Fig36: Command that returns User input back to user as integer value.

E. Question 5 Q:

What GCC command would you use to:

- 1) Only preprocess a file without compiling?
- 2) Generate assembly code but not assemble it?
- 3) Create an object file without linking?

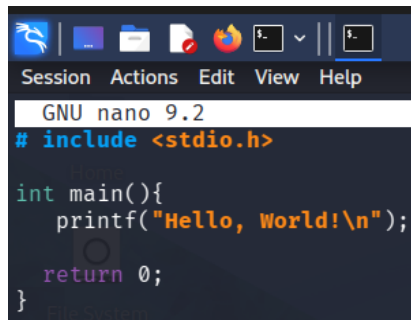
1. Preprocessing:

Preprocessing is the first stage in compilation. It handles preprocessor statements like `#include`, `#define`, and `#if` prior to any actual compilation.

➤ GCC command for preprocessing:

`gcc -E program.c -o program.i`

C. Program c code for processing:

A screenshot of a terminal window with the nano 9.2 text editor. The editor shows a C program with the following code:

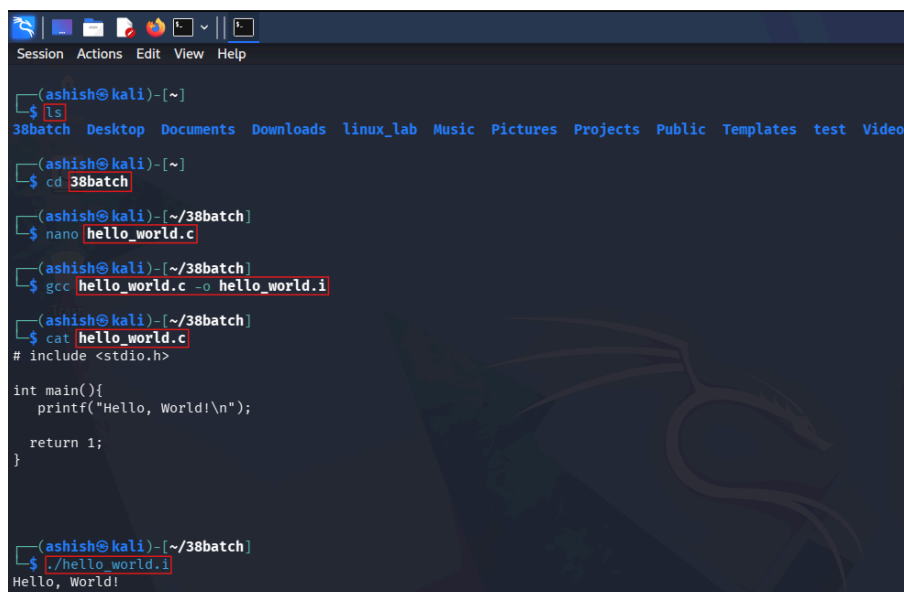
```
#include <stdio.h>

int main(){
    printf("Hello, World!\n");
    return 0;
}
```

Fig37: Program in c for processing state and create object as .i.

B. Step 2: Compile and Run: Command user for Processing state.

Screen short:

A screenshot of a terminal window showing the steps to compile and run a C program. The commands and their outputs are as follows:

```
(ashish@kali)-[~]
$ ls
38batch Desktop Documents Downloads linux_lab Music Pictures Projects Public Templates test Videos

(ashish@kali)-[~]
$ cd 38batch

(ashish@kali)-[~/38batch]
$ nano hello_world.c

(ashish@kali)-[~/38batch]
$ gcc hello_world.c -o hello_world.i

(ashish@kali)-[~/38batch]
$ cat hello_world.c
# include <stdio.h>

int main(){
    printf("Hello, World!\n");
    return 1;
}

(ashish@kali)-[~/38batch]
$ ./hello_world.i
Hello, World!
```

Fig38: Command in C program that returns output to the user from the object.

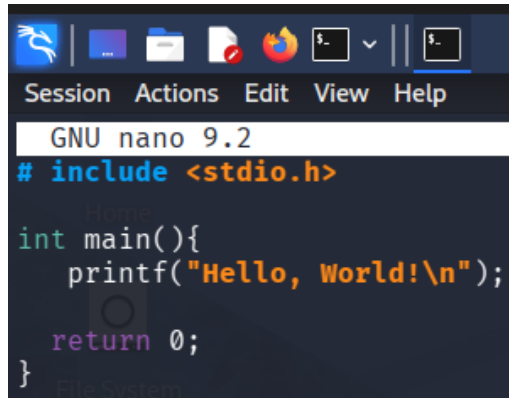
2. Compilation:

The compilation process converts the preprocessed C code into assembly language.

➤ **Command to generate assembly code using GCC:**

`gcc -S program.c -o program.s`

C. Program c code for Compilation:



```
GNU nano 9.2
# include <stdio.h>

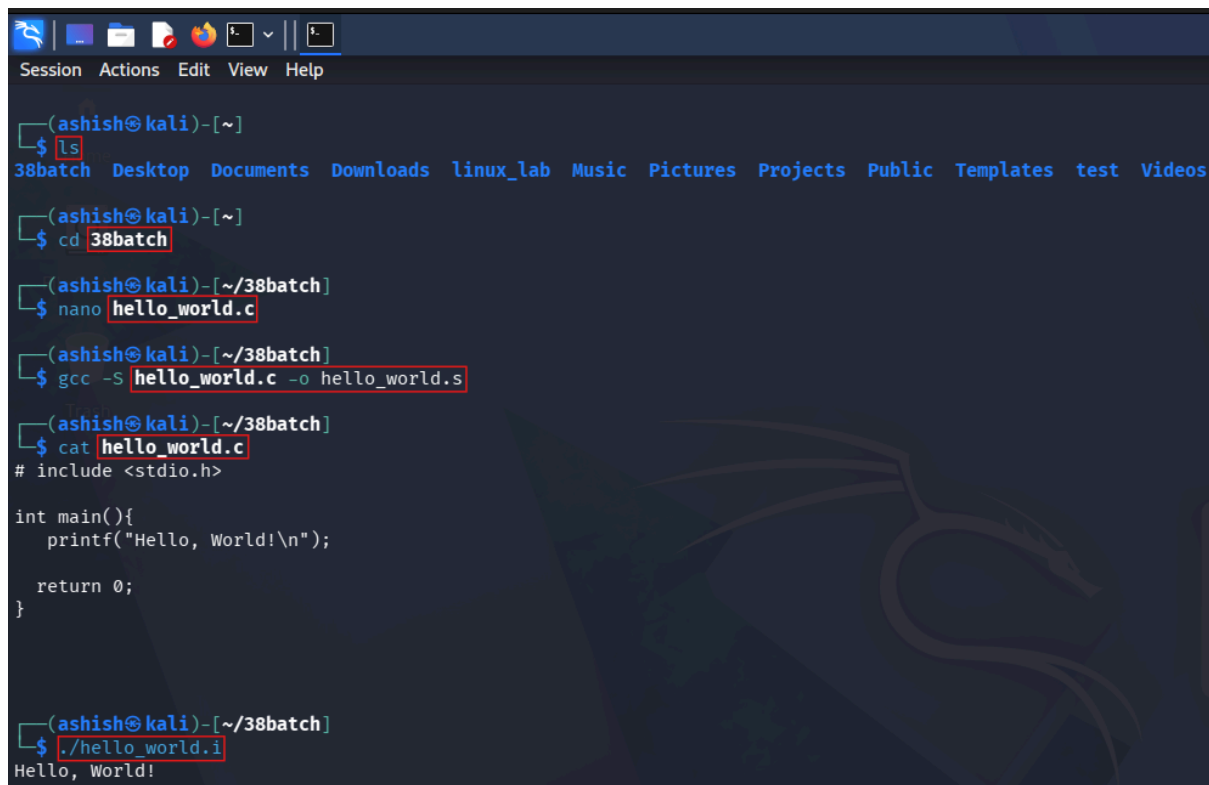
int main(){
    printf("Hello, World!\n");

    return 0;
}
```

Fig39: Program in C to create objects using the `.s` command.

B. Step 2: Compile and Run: Command used for Compilation state.

Screen short:



```
(ashish@kali)-[~]
$ ls
38batch Desktop Documents Downloads linux_lab Music Pictures Projects Public Templates test Videos

(ashish@kali)-[~]
$ cd 38batch

(ashish@kali)-[~/38batch]
$ nano hello_world.c

(ashish@kali)-[~/38batch]
$ gcc -S hello_world.c -o hello_world.s

(ashish@kali)-[~/38batch]
$ cat hello_world.c
# include <stdio.h>

int main(){
    printf("Hello, World!\n");

    return 0;
}

(ashish@kali)-[~/38batch]
$ ./hello_world.i
Hello, World!
```

Fig40: Command that creates objects `-o hello_word.s` for Processing State.

Fig42: Command that Creates objects for processing in C.

F. Question 6 Q: Explain why C is called a "compiled language" and how this differs from interpreted languages.

VI. LAB TASKS COMPLETED:

Task 1: Compiled hello.c with return 0 and verified exit status using echo \$? 2)

Task 2: Modified program to return 1 and observed different exit status 3)

Task 3: Created program accepting user input and displaying it 4)

Task 4: Executed all four compilation stages separately and documented output files

Task 5: Compiled and ran program returning integer value 50

1. C as a Compiled Language:

The reason why C is referred to as a compiled language is that C source code needs to be converted into machine code by the compiler before the program can be run.

The GCC compiler carries out various processes such as pre-processing, compiling, assembling and linking. Once all these processes have been carried out, the program becomes executable by the computer.

➤ **GCC Command:**

gcc hello.c -o hello



Fig43: Program c code for Creation of Object File Using gcc Command.

Summary:

In the command above, the hello.c source code will be compiled into a hello executable file.

C. Program c code for Creation of Object File:

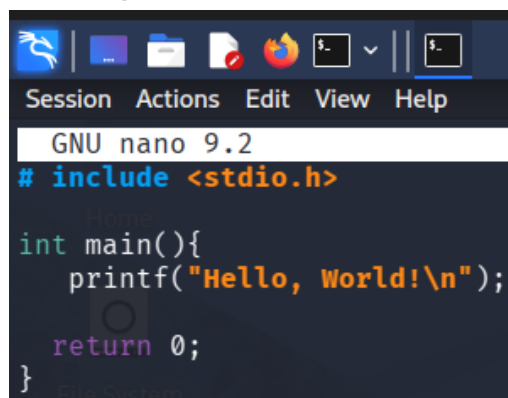
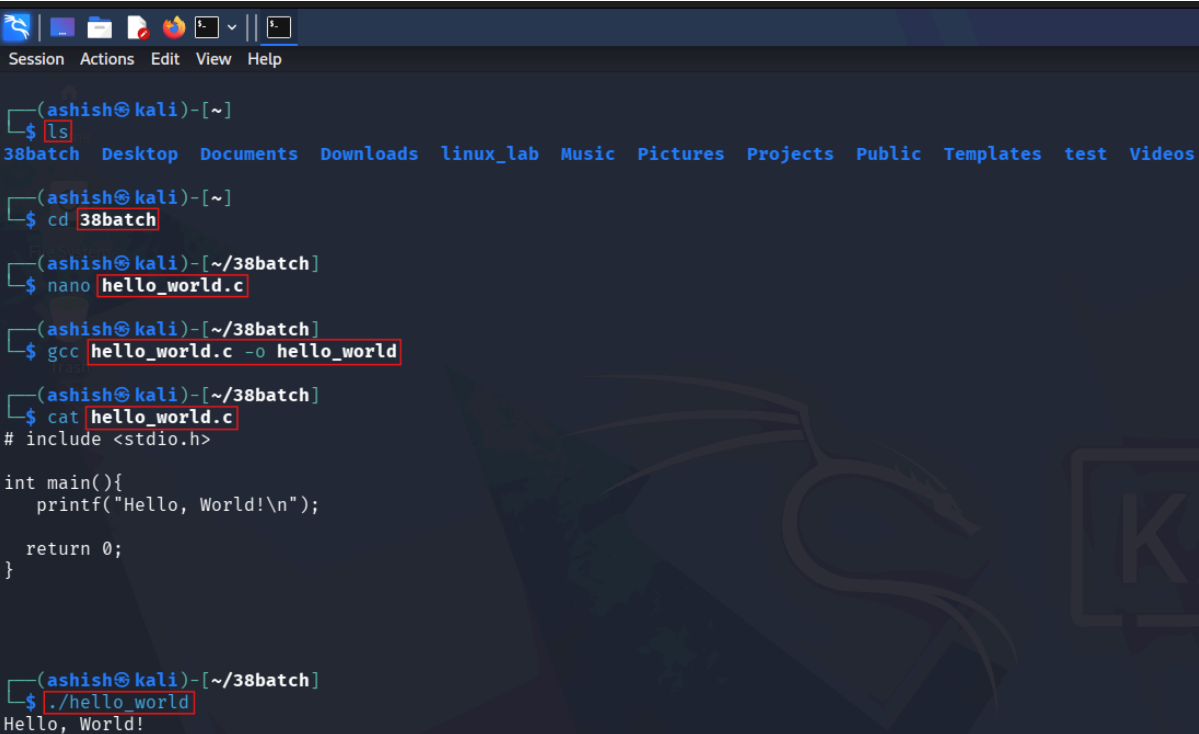


Fig44: Program in C for Creation of Object File.

B. Step 2: Compile and Run: Command used to show how C programs compile.

Screen short:



```
(ashish@kali)-[~]
$ ls
38batch Desktop Documents Downloads linux_lab Music Pictures Projects Public Templates test Videos

(ashish@kali)-[~]
$ cd 38batch

(ashish@kali)-[~/38batch]
$ nano hello_world.c

(ashish@kali)-[~/38batch]
$ gcc hello_world.c -o hello_world

(ashish@kali)-[~/38batch]
$ cat hello_world.c
# include <stdio.h>

int main(){
    printf("Hello, World!\n");

    return 0;
}

(ashish@kali)-[~/38batch]
$ ./hello_world
Hello, World!
```

Fig 45: Command to Compile a C program using GCC.

2. Interpreted Languages:

An interpreted language usually runs source code with the help of an interpreter and does not create a standalone executable file like a conventional compilation method for C code.

For instance, Python code is run by Python interpreter:

`python program.py`

The interpreter runs the Python code.

A. Basic Hello World Program:

The following demonstrates a basic python program which print "Hello World!":

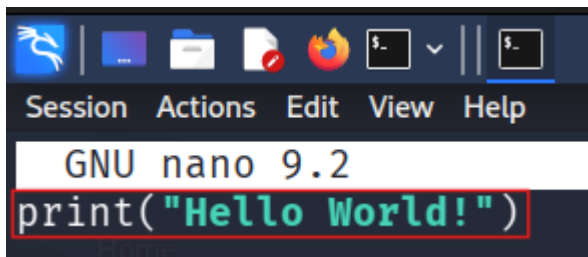


Fig46: Program python.

B. Step 2: Compile and Run: Command to show how python Interpreted language.

Screen short:

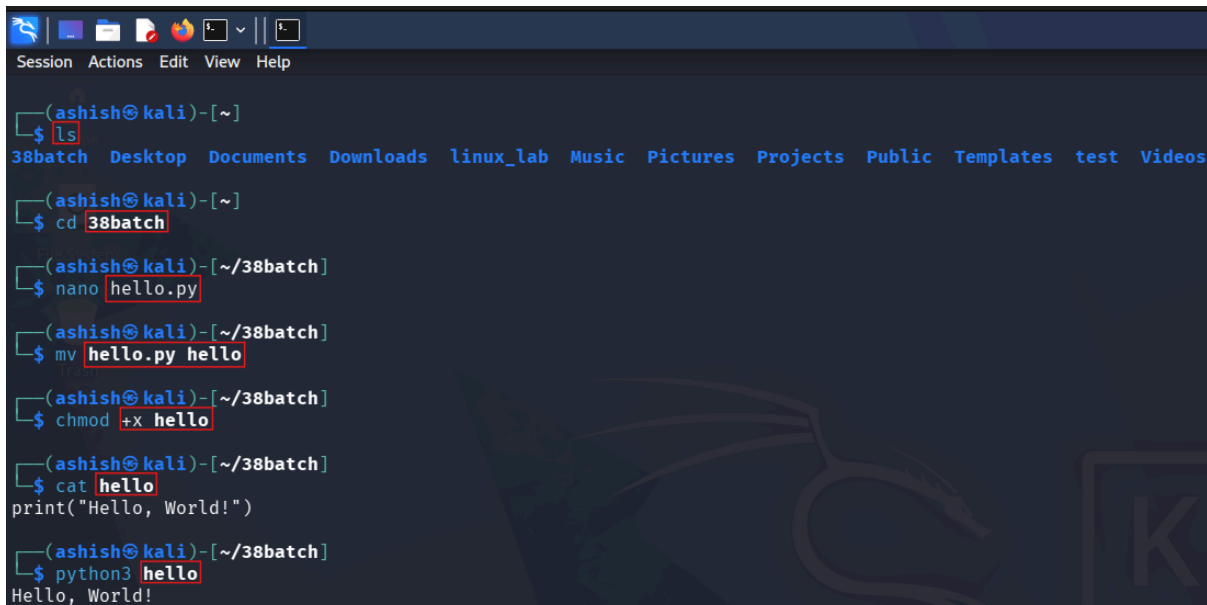


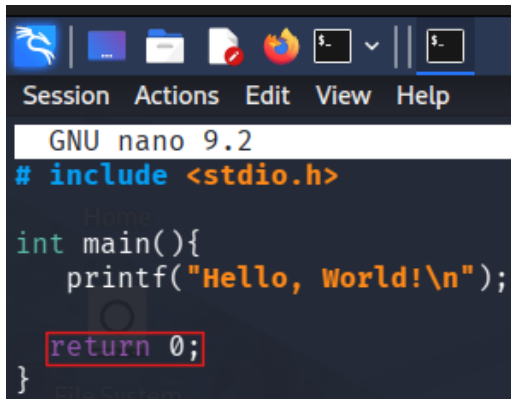
Fig47: Command that returns value in python Code as output.

I. LAB TASKS COMPLETED:

Task 1: Compiled `hello.c` with return 0 and verified exit status using `echo $? 2)`

A. Basic Hello World Program:

The following demonstrates a basic C program structure with preprocessor directives, main function, and return statement:



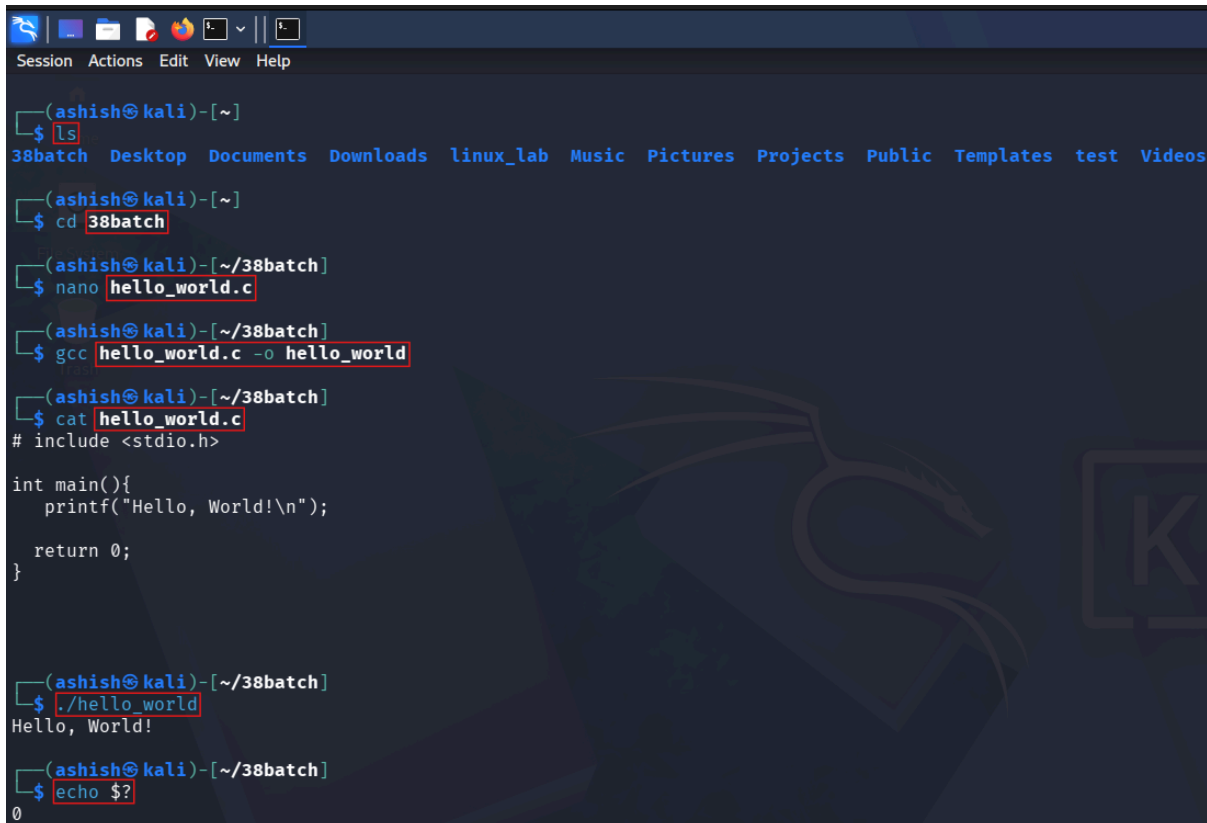
```
GNU nano 9.2
# include <stdio.h>

int main(){
    printf("Hello, World!\n");
    return 0;
}
```

Fig48: Program in C that returns 0.

B. Step 2: Compile and Run: Command That returns 0 as output in C.

Screen short:



```
(ashish@kali)-[~]
$ ls
38batch Desktop Documents Downloads linux_lab Music Pictures Projects Public Templates test Videos

(ashish@kali)-[~]
$ cd 38batch

(ashish@kali)-[~/38batch]
$ nano hello_world.c

(ashish@kali)-[~/38batch]
$ gcc hello_world.c -o hello_world

(ashish@kali)-[~/38batch]
$ cat hello_world.c
# include <stdio.h>

int main(){
    printf("Hello, World!\n");

    return 0;
}

(ashish@kali)-[~/38batch]
$ ./hello_world
Hello, World!

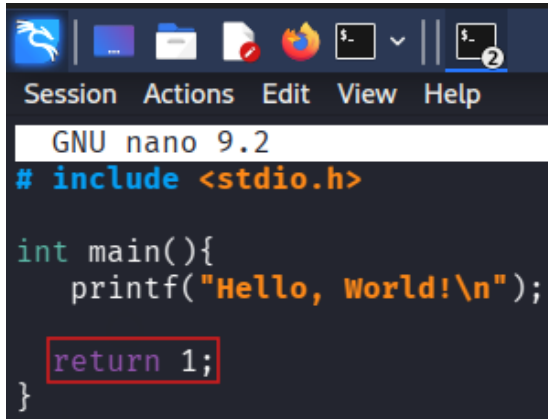
(ashish@kali)-[~/38batch]
$ echo $?
0
```

Fig49: Command that returns 0 as output in c.

Task 2: Modified program to return 1 and observed different exit status 3)

A. Basic Hello World Program:

The following demonstrates a basic C program structure with preprocessor directives, main function, and return statement:



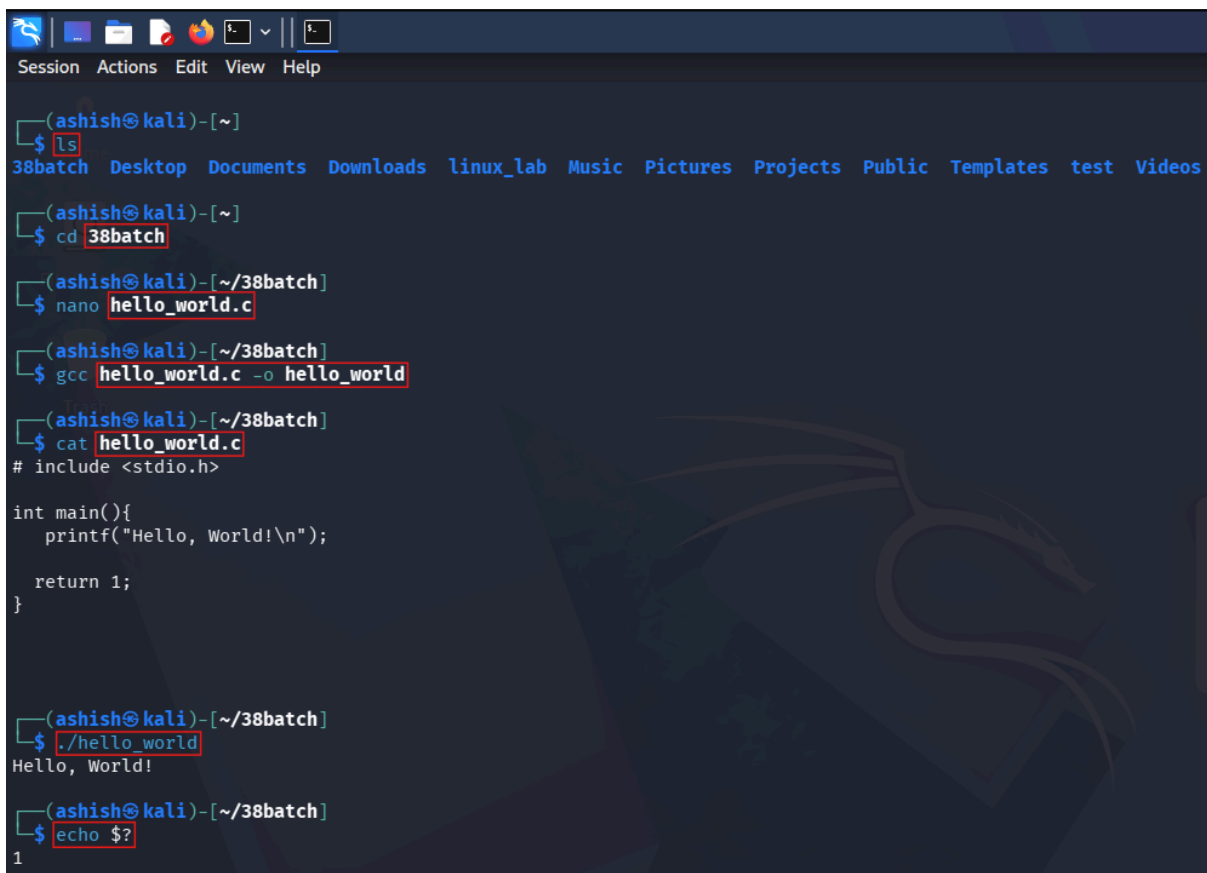
```
GNU nano 9.2
# include <stdio.h>

int main(){
    printf("Hello, World!\n");
    return 1;
}
```

Fig50: C program to return value 1.

B. Step 2: Compile and Run: Command That returns 0 as output in C.

Screen short:



```
(ashish@kali)-[~]
$ ls
38batch Desktop Documents Downloads linux_lab Music Pictures Projects Public Templates test Videos

(ashish@kali)-[~]
$ cd 38batch

(ashish@kali)-[~/38batch]
$ nano hello_world.c

(ashish@kali)-[~/38batch]
$ gcc hello_world.c -o hello_world

(ashish@kali)-[~/38batch]
$ cat hello_world.c
# include <stdio.h>

int main(){
    printf("Hello, World!\n");

    return 1;
}

(ashish@kali)-[~/38batch]
$ ./hello_world
Hello, World!

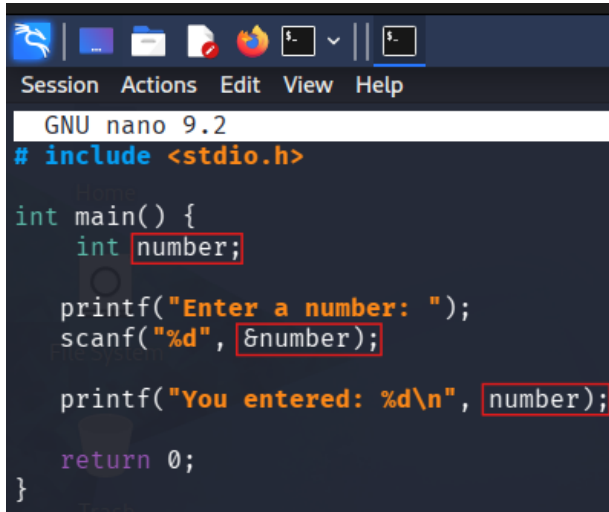
(ashish@kali)-[~/38batch]
$ echo $?
1
```

Fig51: Command that returns 1 as output in c.

Task 3: Created program accepting user input and displaying it 4)

A. Basic return of User Input Program:

The following demonstrates a basic C program to return the user input value:

A screenshot of a terminal window with the nano 9.2 editor open. The editor shows a C program that includes <stdio.h>, defines a main function with an integer variable 'number', prompts the user to enter a number, reads the input, and then prints the entered value. The code is as follows:

```
GNU nano 9.2
# include <stdio.h>

int main() {
    int number;

    printf("Enter a number: ");
    scanf("%d", &number);

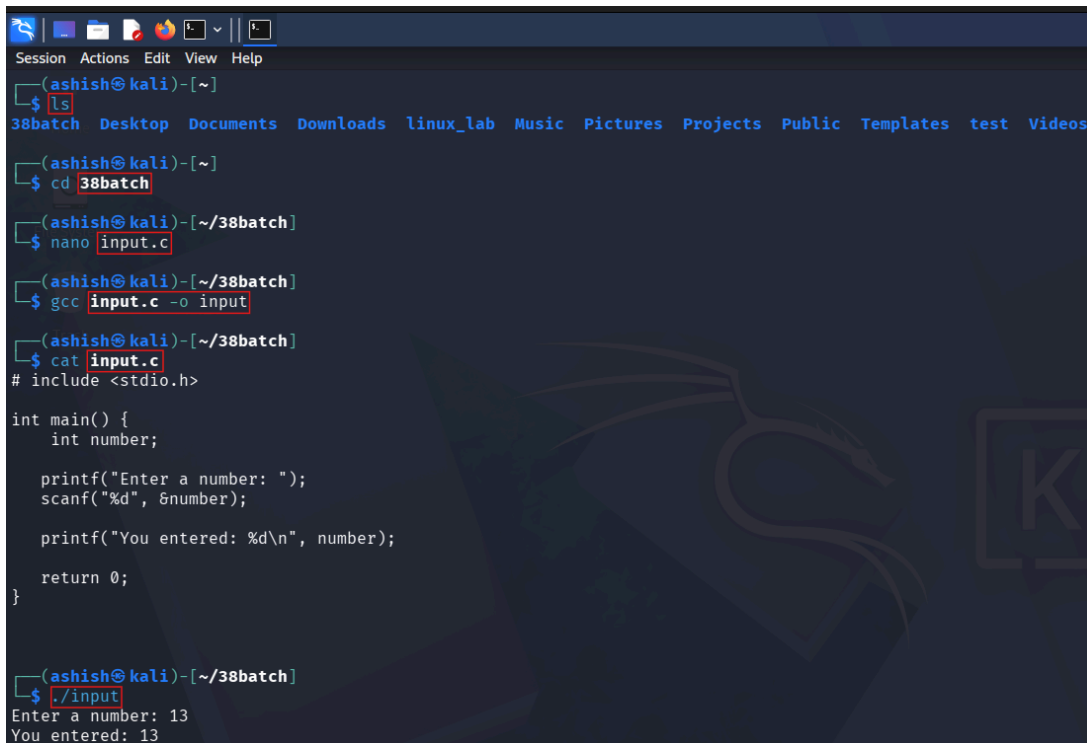
    printf("You entered: %d\n", number);

    return 0;
}
```

Fig52: C program for user input and return the value.

B. Step 2: Compile and Run: Command That takes 13 from the user and returns back to the user.

Screen short:

A screenshot of a terminal window showing the steps to compile and run the C program. The user is in the ~/38batch directory. They list files, navigate to 38batch, create a file named input.c with nano, compile it with gcc, view the source with cat, and finally run the executable with ./input. The output shows the program prompting for a number, receiving 13, and displaying "You entered: 13".

```
(ashish@kali)-[~]
$ ls
38batch Desktop Documents Downloads linux_lab Music Pictures Projects Public Templates test Videos

(ashish@kali)-[~]
$ cd 38batch

(ashish@kali)-[~/38batch]
$ nano input.c

(ashish@kali)-[~/38batch]
$ gcc input.c -o input

(ashish@kali)-[~/38batch]
$ cat input.c
# include <stdio.h>

int main() {
    int number;

    printf("Enter a number: ");
    scanf("%d", &number);

    printf("You entered: %d\n", number);

    return 0;
}

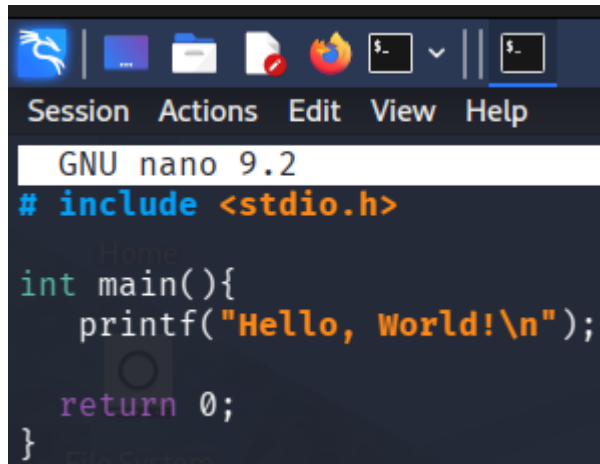
(ashish@kali)-[~/38batch]
$ ./input
Enter a number: 13
You entered: 13
```

Fig53: Command that takes integers as input and returns back to the user.

Task 4: Executed all four compilation stages separately and documented output files:

A. Basic Hello World Program:

The following demonstrates a basic C program structure with preprocessor directives, main function, and return statement:



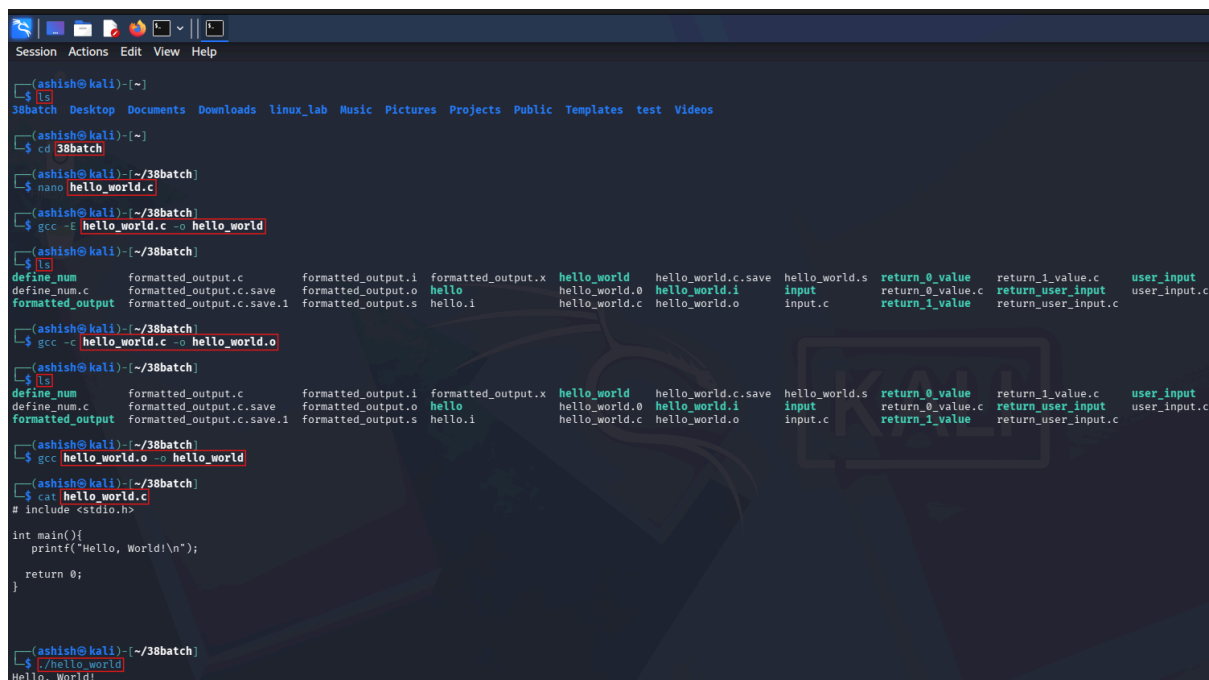
```
GNU nano 9.2
# include <stdio.h>

int main(){
    printf("Hello, World!\n");
    return 0;
}
```

Fig54: C program for user input and return the value.

B. Step 2: Compile and Run: Command that returns output using object on C.

Screen short:



```
(ashish@kali)~[~]
$ ls
38batch Desktop Documents Downloads linux_lab Music Pictures Projects Public Templates test Videos

(ashish@kali)~[~]
$ cd 38batch

(ashish@kali)~/38batch
$ nano hello_world.c

(ashish@kali)~/38batch
$ gcc -E hello_world.c -o hello_world

(ashish@kali)~/38batch
$ ls
define_num      formatted_output.c      formatted_output.i      formatted_output.x      hello_world      hello_world.c.save      hello_world.s      return_0_value      return_1_value.c      user_input
define_num.c    formatted_output.c.save formatted_output.o      hello               hello_world.0         hello_world.i          input.c            return_0_value.c    return_1_value.c      return_user_input.c
formatted_output formatted_output.c.save.1 formatted_output.s      hello.i             hello_world.c         hello_world.o          input.c            return_1_value     return_user_input.c  return_user_input.c

(ashish@kali)~/38batch
$ gcc -c hello_world.c -o hello_world.o

(ashish@kali)~/38batch
$ ls
define_num      formatted_output.c      formatted_output.i      formatted_output.x      hello_world      hello_world.c.save      hello_world.s      return_0_value      return_1_value.c      user_input
define_num.c    formatted_output.c.save formatted_output.o      hello               hello_world.0         hello_world.i          input.c            return_0_value.c    return_1_value.c      return_user_input.c
formatted_output formatted_output.c.save.1 formatted_output.s      hello.i             hello_world.c         hello_world.o          input.c            return_1_value     return_user_input.c  return_user_input.c

(ashish@kali)~/38batch
$ gcc hello_world.o -o hello_world

(ashish@kali)~/38batch
$ cat hello_world.c
# include <stdio.h>

int main(){
    printf("Hello, World!\n");
    return 0;
}

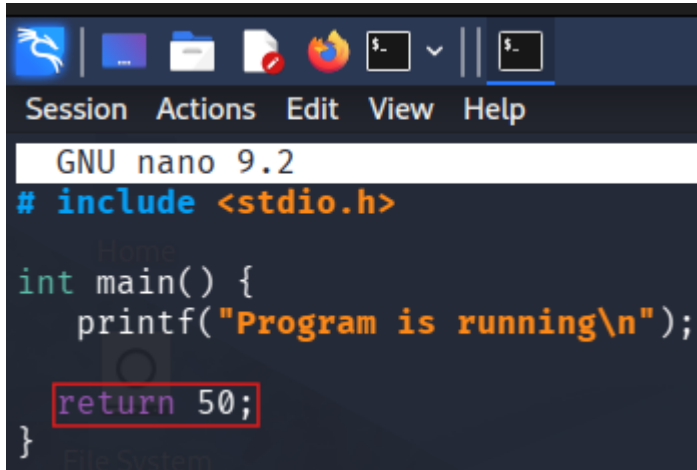
(ashish@kali)~/38batch
$ ./hello_world
Hello, World!
```

Fig55: Command that creates objects and returns output back to the user.

Task 5: Compiled and ran program returning integer value 50

A. Basic C program to returning integer value 50:

The following demonstrates a basic C program structure with preprocessor directives, main function, and return statement:



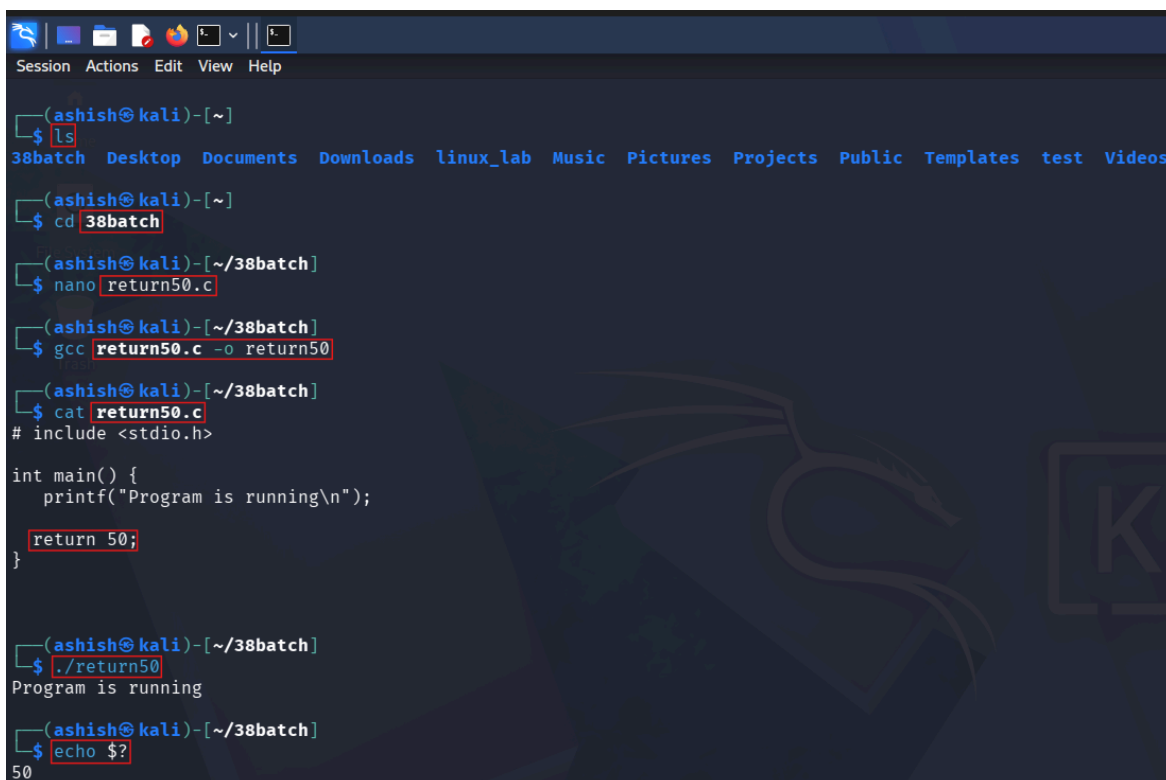
```
GNU nano 9.2
# include <stdio.h>

int main() {
    printf("Program is running\n");
    return 50;
}
```

Fig56: C program code to return integer value 50.

B. Step 2: Compile and Run: Command that returns 50 as output.

Screen short:



```
(ashish@kali)-[~]
$ ls
38batch Desktop Documents Downloads linux_lab Music Pictures Projects Public Templates test Videos

(ashish@kali)-[~]
$ cd 38batch

(ashish@kali)-[~/38batch]
$ nano return50.c

(ashish@kali)-[~/38batch]
$ gcc return50.c -o return50

(ashish@kali)-[~/38batch]
$ cat return50.c
# include <stdio.h>

int main() {
    printf("Program is running\n");
    return 50;
}

(ashish@kali)-[~/38batch]
$ ./return50
Program is running

(ashish@kali)-[~/38batch]
$ echo $?
50
```

Fig57: Step of Program c code to return user input back to user.

➤ **Command table to return user input back to user:**

<u>FINAL LAB TASK SUMMARY</u>		
<u>Task</u>	<u>Main Command</u>	<u>Expected Result</u>
1.	gcc hello.c -o hello	echo \$? -> 0
2.	gcc hello.c -o hello	echo \$? -> 1
3.	gcc input.c -o input	Displays entered number
4.	gcc -E, gcc -S, gcc -c, gcc	.i, .s, .o, executable
5.	gcc return50.c -o return50	echo \$? -> 50

Fig58: Step of Program c code to return user input back to user.

VII. SCREENSHOTS AND EVIDENCE Note:

Students should include screenshots of:

- Terminal showing GCC installation and version check
- Each compilation stage (.i, .s, .o files)
- Program execution with different return values
- Exit status verification using echo \$?

VII. SCREENSHOTS AND EVIDENCE:

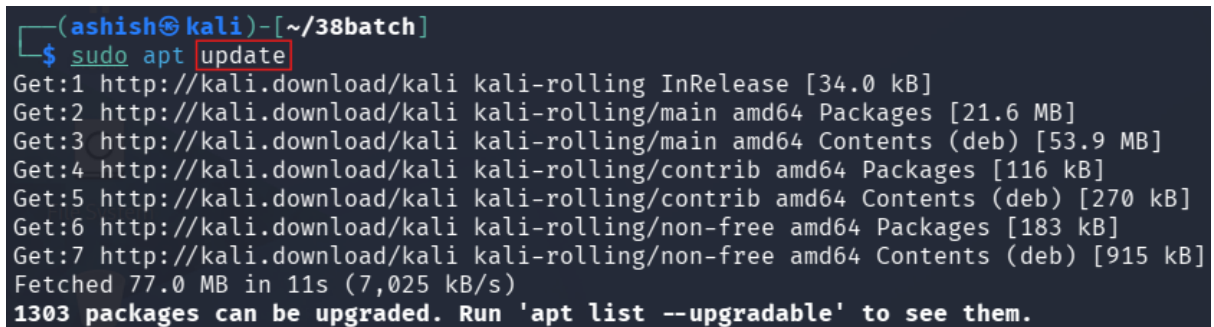
The following are screenshots which are provided as evidence of completed tasks from the laboratory session on compilation of C programs using GCC under Linux.

1. Update Your Package List:

The best thing to do is always to update your package list. Type in your terminal:

Command:

➤ **sudo apt update.**



```
(ashish@kali)-[~/38batch]
$ sudo apt update
Get:1 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 Packages [21.6 MB]
Get:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [53.9 MB]
Get:4 http://kali.download/kali kali-rolling/contrib amd64 Packages [116 kB]
Get:5 http://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [270 kB]
Get:6 http://kali.download/kali kali-rolling/non-free amd64 Packages [183 kB]
Get:7 http://kali.download/kali kali-rolling/non-free amd64 Contents (deb) [915 kB]
Fetched 77.0 MB in 11s (7,025 kB/s)
1303 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

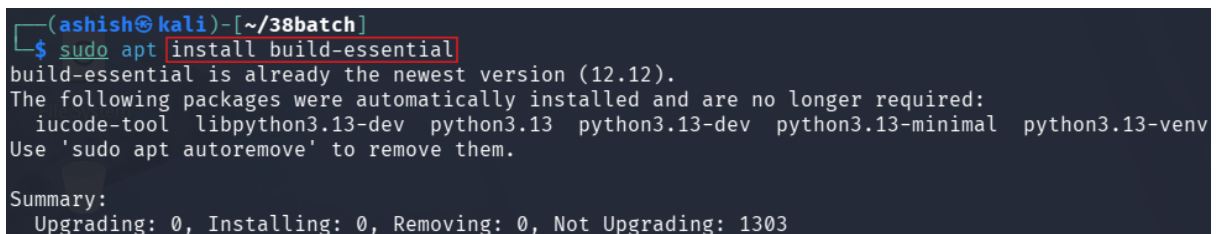
Fig 59: Update of linux version.

2. Installation of GCC and Other Build Essentials:

It is suggested that the installation be done using the build-essential package. The build-essential is a metapackage that contains the GCC, g++ for C++, Make, and other essential libraries.

Command:

➤ **sudo apt install build-essential.**



```
(ashish@kali)-[~/38batch]
$ sudo apt install build-essential
build-essential is already the newest version (12.12).
The following packages were automatically installed and are no longer required:
  iucode-tool libpython3.13-dev python3.13 python3.13-dev python3.13-minimal python3.13-venv
Use 'sudo apt autoremove' to remove them.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 1303
```

Fig 60: Installation of build-essential.

3. Installation of GCC:

The screen shot below indicates that GCC is properly installed and shows the installed version of GCC.

Command to install gcc:

➤ **sudo apt install gcc.**

[illegible]

Fig 61: Installation of GCC.

4. Check for Version:

The screen shot below indicates the version of GCC.

Command used to check gcc version:

➤ **gcc --version.**

```
(ashish@kali)-[~/38batch]
$ gcc --version
gcc (Debian 15.2.0-17) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

Fig 62: Check version of GCC.

5. Preprocessing Stage:

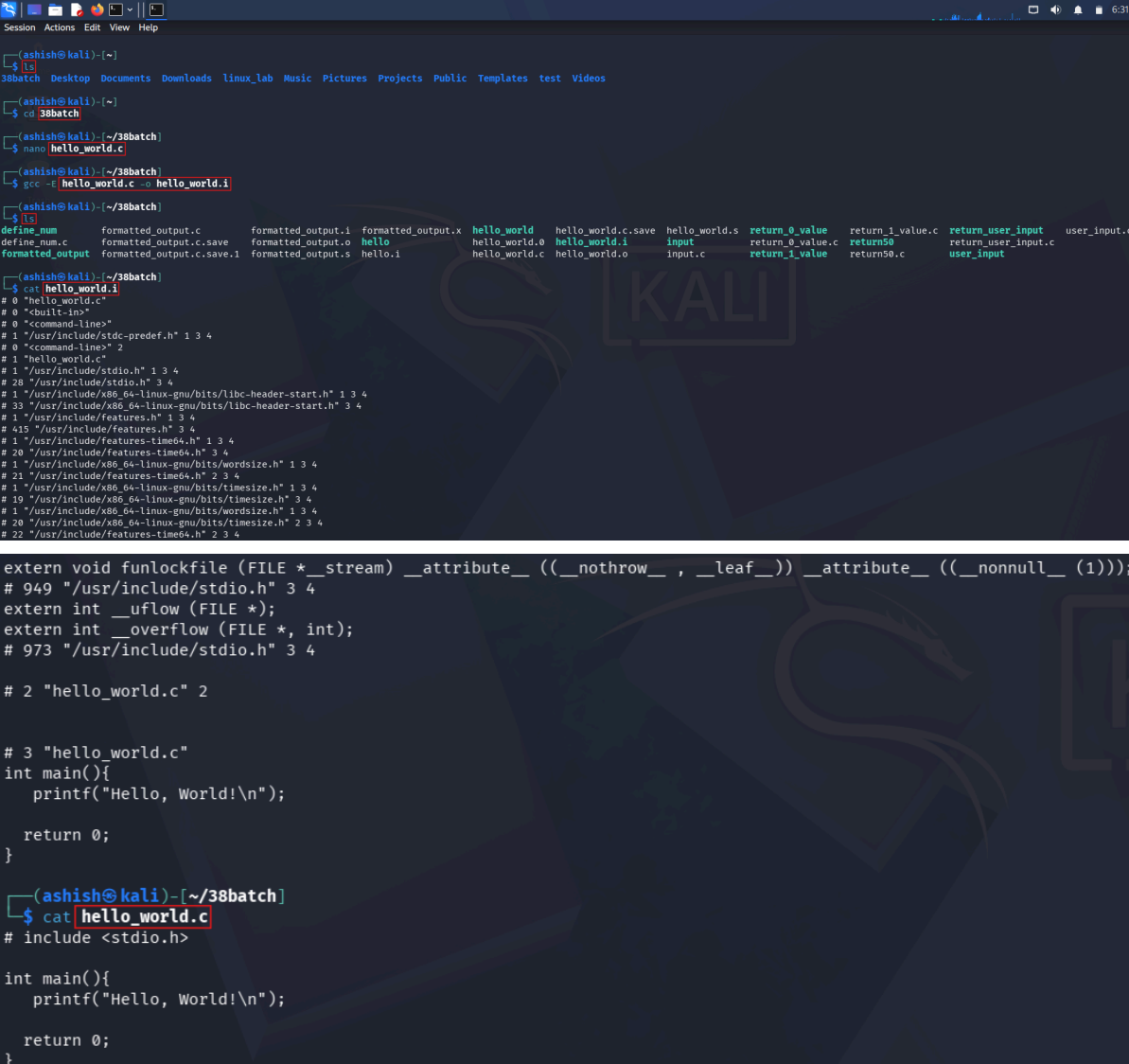
The screenshot shows the preprocessing stage of the C compilation process and the creation of the .i file.

Command used:

➤ gcc -E hello.c -o hello.i

B. Step 2: Compile and Run: Processing Stage.

Screen short:



```
(ashish@kali)~$ ls
38batch Desktop Documents Downloads linux_lab Music Pictures Projects Public Templates test Videos

(ashish@kali)~$ cd 38batch

(ashish@kali)~/38batch$ nano hello_world.c

(ashish@kali)~/38batch$ gcc -E hello_world.c -o hello_world.i

(ashish@kali)~/38batch$ ls
define_num      formatted_output.c      formatted_output.i      formatted_output.o      formatted_output.s      formatted_output.x      hello_world            hello_world.c.save     hello_world.s          return_0_value         return_1_value.c       return_user_input      user_input.c
define_num.c    formatted_output.c.save  formatted_output.o      hello                  hello_world.0          hello_world.i          input.c               return_0_value.c       return_1_value         return50.c             return_user_input.c    user_input

(ashish@kali)~/38batch$ cat hello_world.i
# 0 "hello_world.c"
# 0 <built-in>
# 0 <command-line>
# 1 "/usr/include/stdc-predef.h" 1 3 4
# 0 <command-line> 2
# 1 "hello_world.c"
# 1 "/usr/include/stdio.h" 1 3 4
# 28 "/usr/include/stdio.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 1 3 4
# 33 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 3 4
# 1 "/usr/include/features.h" 1 3 4
# 415 "/usr/include/features.h" 3 4
# 1 "/usr/include/features-time64.h" 1 3 4
# 20 "/usr/include/features-time64.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 21 "/usr/include/features-time64.h" 2 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 1 3 4
# 19 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 20 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 2 3 4
# 22 "/usr/include/features-time64.h" 2 3 4

extern void funlockfile (FILE *_stream) __attribute__ ((__nothrow__ , __leaf__)) __attribute__ ((__nonnull__ (1)));
# 949 "/usr/include/stdio.h" 3 4
extern int __uflow (FILE *);
extern int __overflow (FILE *, int);
# 973 "/usr/include/stdio.h" 3 4

# 2 "hello_world.c" 2

# 3 "hello_world.c"
int main(){
    printf("Hello, World!\n");

    return 0;
}

(ashish@kali)~/38batch$ cat hello_world.c
# include <stdio.h>

int main(){
    printf("Hello, World!\n");

    return 0;
}
```

Fig 63: Preprocessed file hello_world.i.

3. Compilation Stage:

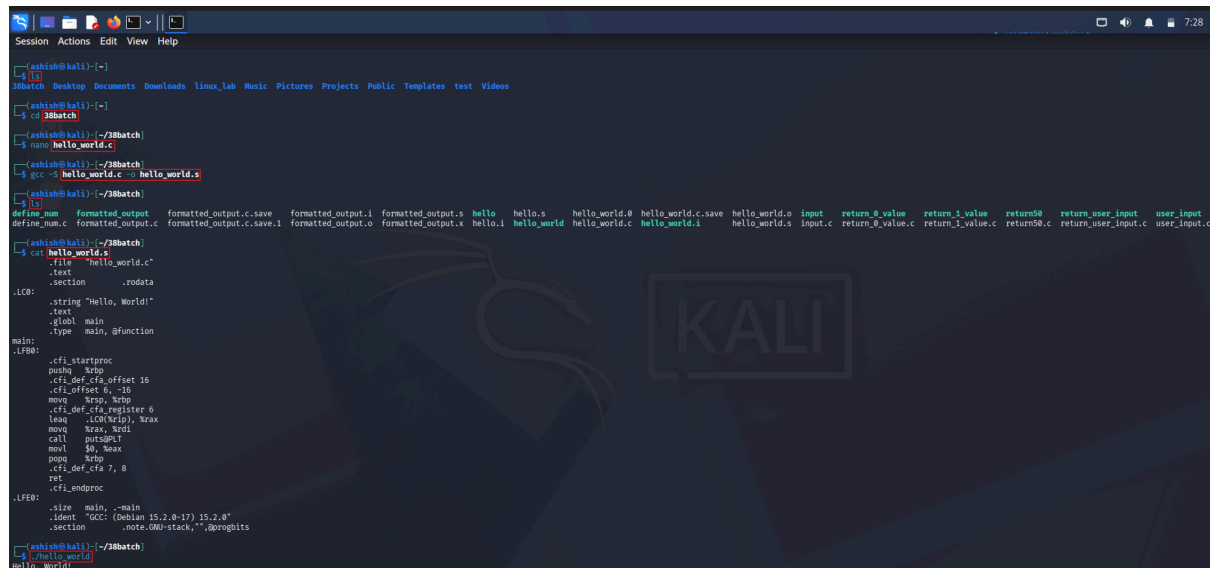
The screenshot shows the generation of assembly code from the C program.

Command used:

➤ `gcc -S hello.c -o hello.s`

B. Step 2: Compile and Run: Compilation Stage.

Screen short:



```
ashish@kali:~$ ls
ashish@kali:~$ cd /tmp
ashish@kali:~/tmp$ gcc -S hello.c -o hello.s
ashish@kali:~/tmp$ cat hello.s
.file "hello_world.c"
.text
.section .rodata
.LC0:
.string "Hello, World!"
.text
.globl main
.type main, @function
main:
.LFB0:
.cfi_startproc
pushl %ebp
.cfi_def_cfa_offset 16
.cfi_offset 0, 16
movl %esp, %ebp
.cfi_def_cfa_register 6
leal -4(%ebp), %eax
movl %eax, %edi
call puts@PLT
movl $0, %eax
popl %ebp
.cfi_def_cfa 7, 8
ret
.cfi_endproc
.LFE0:
.size main, .-main
.ident "gcc (Debian 15.2.0-17) 15.2.0"
.section .note.GNU-stack,"",@progbits
ashish@kali:~/tmp$ ./hello_world
Hello, World!
```

Fig 64: Generated assembly file hello.s.

4. Assembly Stage:

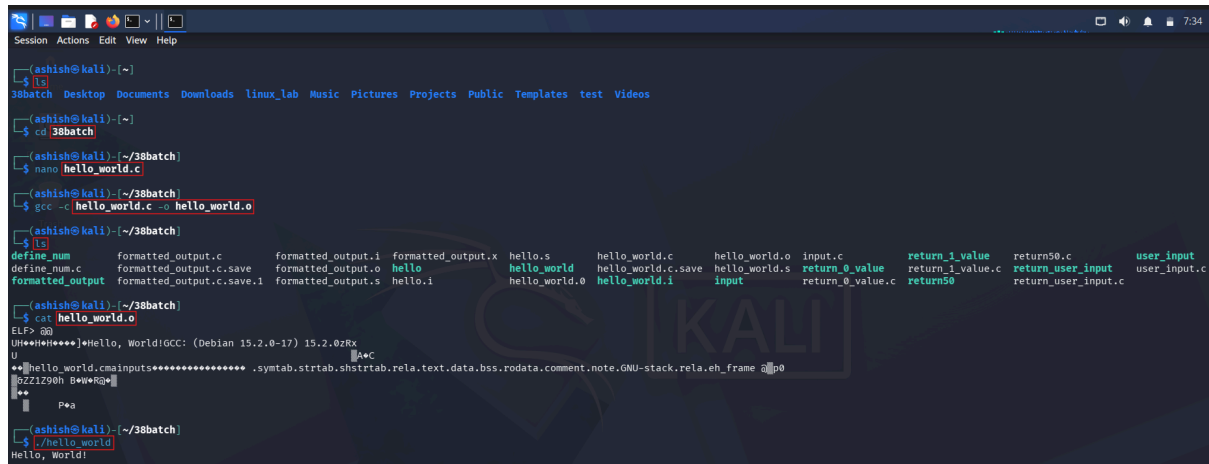
The screenshot shows the creation of the object file without linking.

Command used:

➤ `gcc -c hello.c -o hello.o`

B. Step 2: Compile and Run: Assembly Stage.

Screen short:



```
(ashish@kali):~$ ls
38batch Desktop Documents Downloads linux_lab Music Pictures Projects Public Templates test Videos

(ashish@kali):~$ cd 38batch

(ashish@kali):~/38batch$ nano hello_world.c

(ashish@kali):~/38batch$ gcc -c hello_world.c -o hello_world.o

(ashish@kali):~/38batch$ ls
define_num      formatted_output.c      formatted_output.i      formatted_output.x      hello.s      hello_world.c      hello_world.o      input.c      return_1_value      return50.c      return_user_input      user_input
define_num.c    formatted_output.c.save formatted_output.o      hello          hello_world      hello_world.c.save  hello_world.s      return_0_value      return_1_value.c  return50.o      return_user_input.c  user_input.c
formatted_output formatted_output.c.save1 formatted_output.s      hello.i        hello_world.o    hello_world.i      input          return_0_value.c  return50          return_user_input.c  return_user_input.c

(ashish@kali):~/38batch$ cat hello_world.o
ELF 32
U0000000000000000Hello, World!GCC: (Debian 15.2.0-17) 15.2.0:Rx
U
**hello_world.c:maininputs***** .symtab.strtab.shstrtab.rela.text.data.bss.rodata.comment.note.GNU-stack.rela.eh_frame 0000
00221290h 00000000
***
P000

(ashish@kali):~/38batch$ ./hello_world
Hello, World!
```

Fig 65: Generated object file hello.o For Assembly Stage.

5. Program Execution with Return Value 0:

The screenshot shows the execution of the C program with return 0; and the verification of the exit status.

Commands used:

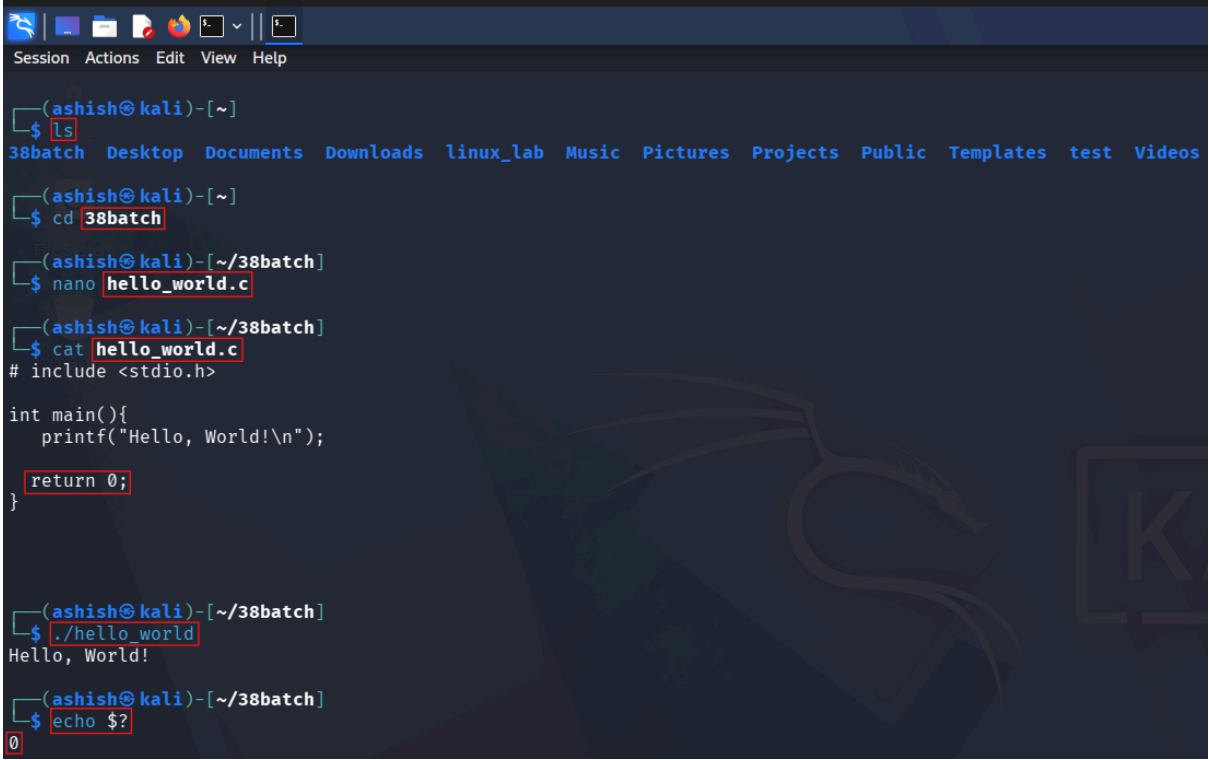
- ./hello_world
- echo \$?

Expected exit status:

0

B. Step 2: Compile and Run: Program Execution With Return Value 0.

Screen short:



```
(ashish@kali)-[~]
$ ls
38batch Desktop Documents Downloads linux_lab Music Pictures Projects Public Templates test Videos

(ashish@kali)-[~]
$ cd 38batch

(ashish@kali)-[~/38batch]
$ nano hello_world.c

(ashish@kali)-[~/38batch]
$ cat hello_world.c
# include <stdio.h>

int main(){
    printf("Hello, World!\n");
    return 0;
}

(ashish@kali)-[~/38batch]
$ ./hello_world
Hello, World!

(ashish@kali)-[~/38batch]
$ echo $?
0
```

Fig 66: Program execution and exit status 0.

6. Program Execution with Return Value 1

The screenshot shows the modified program with return 1; and the resulting exit status.

Commands used:

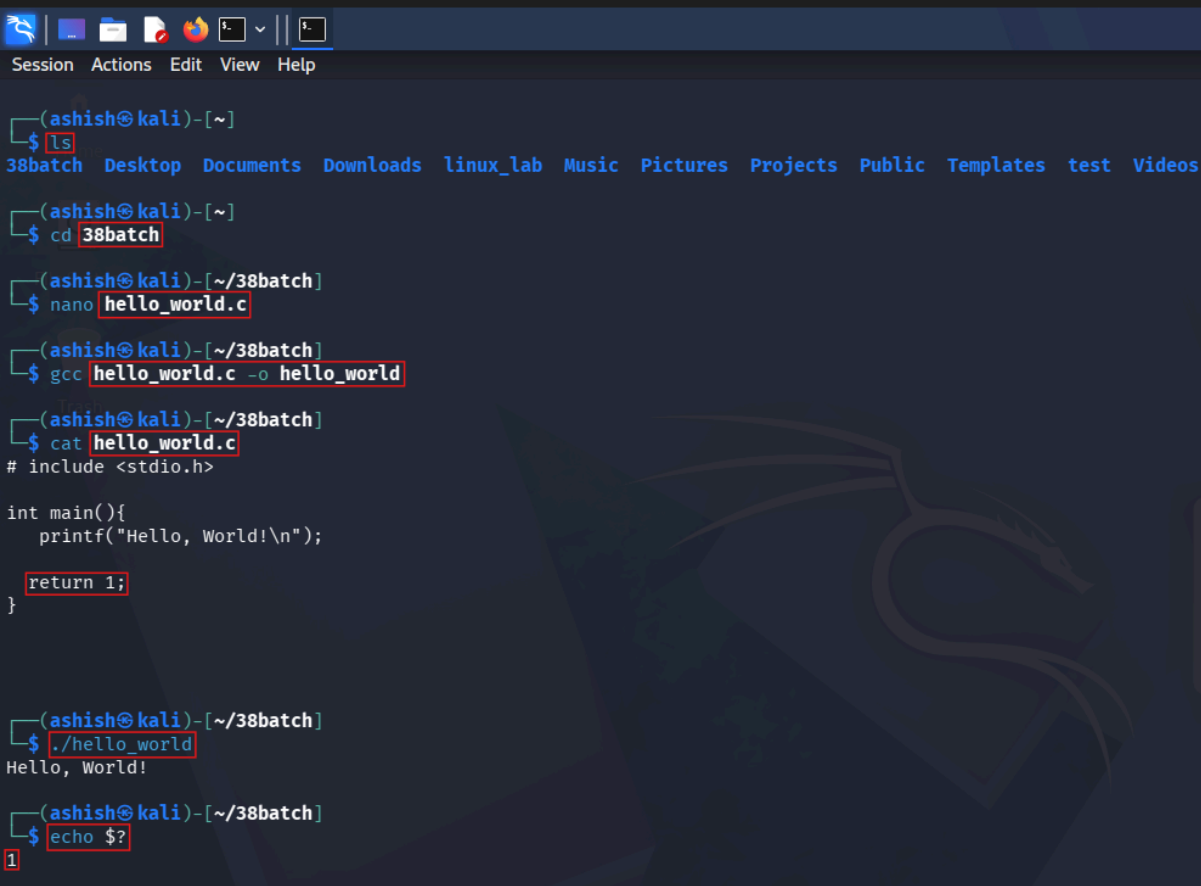
- ./hello
- echo \$?

Expected exit status:

1

B. Step 2: Compile and Run: Program Execution with Return Value 1.

Screen short:



```
(ashish@kali)-[~]
$ ls
38batch Desktop Documents Downloads linux_lab Music Pictures Projects Public Templates test Videos

(ashish@kali)-[~]
$ cd 38batch

(ashish@kali)-[~/38batch]
$ nano hello_world.c

(ashish@kali)-[~/38batch]
$ gcc hello_world.c -o hello_world

(ashish@kali)-[~/38batch]
$ cat hello_world.c
# include <stdio.h>

int main(){
    printf("Hello, World!\n");

    return 1;
}

(ashish@kali)-[~/38batch]
$ ./hello_world
Hello, World!

(ashish@kali)-[~/38batch]
$ echo $?
1
```

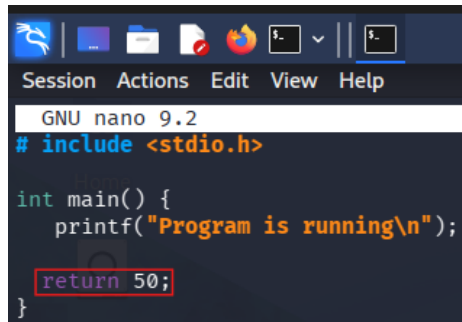
Fig 67: Program execution and exit status 1.

7. Program Returning Integer Value 50:

The screenshot shows the execution of a program containing return 50; and the verification of its exit status.

A. Basic C program to returning integer value 50:

The following demonstrates a basic C program structure with preprocessor directives, main function, and return statement:



```
GNU nano 9.2
# include <stdio.h>

int main() {
    printf("Program is running\n");
    return 50;
}
```

Fig56: C program code to return integer value 50 to user.

Commands used:

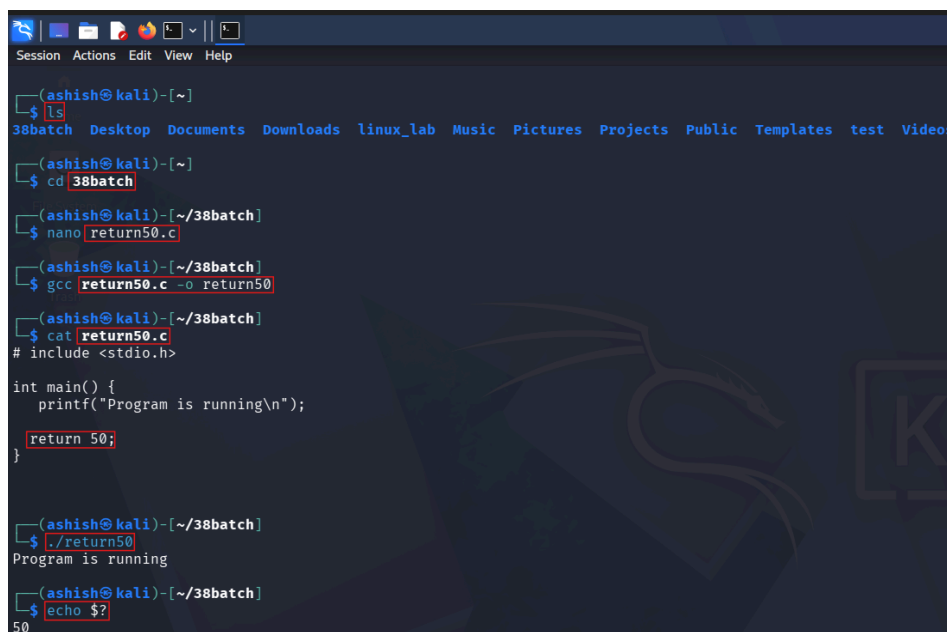
- ./return50
- echo \$?

Expected exit status:

50

B. Step 2: Compile and Run: Command that returns 50 as output in C.

Screen short:



```
(ashish@kali)-[~]
└─$ ls
38batch Desktop Documents Downloads linux_lab Music Pictures Projects Public Templates test Videos

(ashish@kali)-[~]
└─$ cd 38batch

(ashish@kali)-[~/38batch]
└─$ nano return50.c

(ashish@kali)-[~/38batch]
└─$ gcc return50.c -o return50

(ashish@kali)-[~/38batch]
└─$ cat return50.c
# include <stdio.h>

int main() {
    printf("Program is running\n");
    return 50;
}

(ashish@kali)-[~/38batch]
└─$ ./return50
Program is running

(ashish@kali)-[~/38batch]
└─$ echo $?
50
```

Fig57: Program execution and exit status 50.

8. User Input Program:

The screenshot shows the C program accepting an integer from the user and displaying the entered value.

Command used:

➤ `./input`

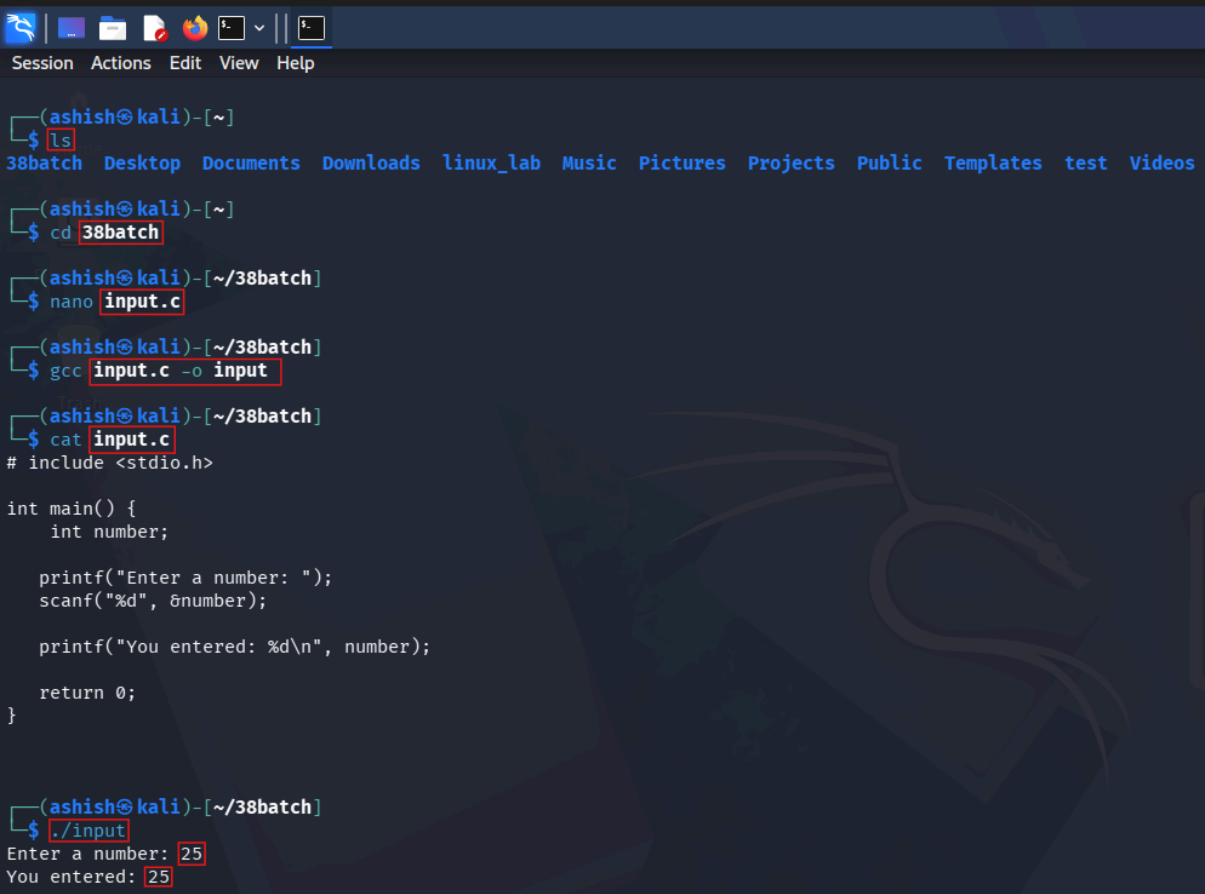
Example output:

Enter a number: 25

You entered: 25

B. Step 2: Compile and Run: User Input Program To return integer back to user.

Screen short:



```
(ashish@kali)-[~]
$ ls
38batch Desktop Documents Downloads linux_lab Music Pictures Projects Public Templates test Videos

(ashish@kali)-[~]
$ cd 38batch

(ashish@kali)-[~/38batch]
$ nano input.c

(ashish@kali)-[~/38batch]
$ gcc input.c -o input

(ashish@kali)-[~/38batch]
$ cat input.c
# include <stdio.h>

int main() {
    int number;

    printf("Enter a number: ");
    scanf("%d", &number);

    printf("You entered: %d\n", number);

    return 0;
}

(ashish@kali)-[~/38batch]
$ ./input
Enter a number: 25
You entered: 25
```

Fig 58: Program accepting and displaying user input as 25 and returns back to user.

CONCLUSION:

The above laboratory activity gave an opportunity to learn about the basics of C programming language, which include understanding how programs are structured and compiled, as well as the use of the GCC compiler. It is necessary for the purpose of systems programming and operating system programming.

Four phases of the compilation process illustrate how source code gets converted into machine code.